

DepthWizard

Complete Technical Reference

Every component of the system, written down in one place: the problem statement verbatim, the full technology stack with versions, the model architecture down to individual layers, every dataset, every hyperparameter, every measured result with the protocol it was measured under, the deployment, the repository map, the negative results, the limitations, and the literature. Nothing summarised away.

This document is a knowledge base. It is written to be read by a person or fed to an assistant and questioned. Facts are stated with their source and their measurement conditions, so an answer drawn from this document can always be traced back to how it was obtained.

PROBLEM STATEMENT

SIH26175 — DepthWizard: Single-View Height Estimation and 3D Flythrough

THEME & CATEGORY

Disaster Management · Software

SHIPPING MODEL

run02 / best.pt, epoch 11, Depth Anything V2 Small, 24.79 M parameters

REPOSITORY

D:\sih2026\depthwizard — 19,713 lines of Python across 62 modules

ORGANISATION

Indian Space Research Organisation — Space Applications Centre

LIVE DEPLOYMENT

<https://project5.zaidansari.tech> — verified HTTP 200 on 6 Sep 2026

HEADLINE ACCURACY

3.667 m per-building RMSE against airborne LiDAR, 80 held-out tiles

COMPILED

6 September 2026 — every figure re-verified against the repository on that date

Sixteen parts

Parts 1–3 are what the project is. Parts 4–8 are how it works. Parts 9–11 are what it produces and where it runs. Parts 12–16 are the supporting record: the code map, what failed, what is broken, the sources, and the words.

WHAT IT IS

- 1 The problem statement, verbatim from the SIH portal
- 2 What the system does, end to end
- 3 The complete technology stack, with versions and reasons

HOW IT WORKS

- 4 The model — architecture, parameters, heads, initialisation
- 5 The loss functions and what each term buys
- 6 The data — every dataset, every split, every measured statistic
- 7 Training — hyperparameters, all six runs, thermals
- 8 Inference — sliding window, TTA, fusion, auto-zoom, DEM anchoring, GCP

WHAT IT PRODUCES

- 9 Results — every measured number with its protocol
- 10 The 3D viewer — design rules, features, standalone build
- 11 Deployment — Azure, DNS, limits, and the constraints that shaped it

THE RECORD

- 12 Repository map — every file and what it is for
- 13 Negative results — everything that was tried and failed
- 14 Limitations, known bugs, and unfinished work
- 15 References, licences, and command reference
- 16 Glossary — every term and abbreviation

HOW TO READ THE NUMBERS IN THIS DOCUMENT

Every accuracy figure carries the protocol that produced it, because figures from different protocols are not comparable and mixing them is the most common way to state something false about this project. There are four protocols in use and they are defined in Part 9. Where a number has no protocol attached, it is a property of the code or the hardware rather than a measurement of accuracy.

The problem statement

Scraped verbatim from sih.gov.in/sih2026PS on 6 September 2026. This is ISRO's own wording. Where any other document in the project disagrees with this, this wins.

1.1 Identification

FIELD	VALUE
Problem Statement ID	26175 (written SIH26175)
Title	DepthWizard - Single-View Height Estimation and 3D Flythrough
Organisation	Indian Space Research Organisation (ISRO)
Department	Department of Space / Indian Space Research Organisation
Category	Software
Theme	Disaster Management
Submissions at scrape time	0 of 500, as of 6 Sep 2026
Deadline	30 September 2026
Reference repository	github.com/IMG-PROCESS-SAC/SIH-DepthWizard-2026
Recommended dataset	GAMUS — huggingface.co/datasets/earthflow/GAMUS
Our team	Dev Up, Team ID 47

A DATE CORRECTION THAT MATTERS

Three internal documents in this project (PLAN.md, README.md and the original team brief) say the deadline is 20 September 2026. The portal prints **30 September 2026** in all 233 problem-statement rows, with no other date anywhere on the page. The portal is right.

1.2 Background, as ISRO wrote it

“Accurate Digital Elevation Models (DEMs) and Digital Surface Models (DSMs) are fundamental to urban planning, disaster management, and military reconnaissance. Traditionally, elevation data is acquired through stereo-imaging pairs, LiDAR, or Interferometric Synthetic Aperture Radar (InSAR). These approaches can be cost-prohibitive, dependent on specific sensor availability, and computationally intensive. Single-view height estimation offers an agile alternative, but foundational monocular depth models are trained largely on natural egocentric imagery and predict relative depth. When applied to remote sensing, they face domain gaps, structural variations, and a lack of absolute-scale mapping. Converting relative depth into metric elevation remains a critical challenge, alongside the operational need to transform static elevation profiles into interactive 3D assets that can be navigated in real time.”

1.3 Description, as ISRO wrote it

“Develop an end-to-end software pipeline that transforms single-view optical RGB remote-sensing images into high-precision elevation maps. The framework must support both non-georeferenced and georeferenced imagery.”

- **Non-Georeferenced RGB Imagery** (for example, PNG or JPG): produce a **Relative Digital Surface Model (rDSM)** for images without spatial metadata.
- **Georeferenced RGB Imagery** (for example, GeoTIFF): produce an **Absolute Digital Surface Model (DSM)** with metric height values for images containing coordinate-system metadata.

“The solution should use a pre-trained monocular depth-estimation backbone to generate initial relative-depth maps. For georeferenced imagery, a lower-resolution DEM source such as SRTM or a limited set of Ground Control Points may be used to map scale-agnostic depth features to absolute metric elevations. For non-georeferenced imagery, relative height may be used directly in the visualization stage.”

“After computing the elevation map, the system should project the original optical image onto a generated 3D terrain mesh and integrate the result with a rendering engine such as Unity, Three.js, or Babylon.js. The interface should support seamless first-person navigation and analysis of structural heights and slopes from arbitrary aerial perspectives.”

1.4 The three key milestones

Elevation Extraction	Use a robust pre-trained monocular depth model to extract geometric and structural representations from single-view optical imagery.
Scale Calibration	Develop a module that converts relative depth to absolute height using scene-level statistics, low-resolution DEMs, semantic priors, or minimal Ground Control Points for georeferenced inputs.
Visualization Layer	Build an immersive, preferably interactive, rendering pipeline that converts the optical texture and derived depth map into a navigable 3D environment deployable as a standalone application.

1.5 Evaluation criteria – this drives every decision in the project

WEIGHT	CRITERION, AS WRITTEN
50%	DSM Estimation — Accuracy and Validation. Evaluate RMSE, MAE, and correlation against LiDAR or reference data, including performance stability across urban, sparse, hilly, and forested landscapes.
50%	Visualization — Rendering Quality and User Experience. Assess projection accuracy, visual fidelity, navigability of the 3D flythrough, interface intuitiveness, software stability, and successful standalone deployment.

Half the marks are the front end. This is the single most consequential fact in the problem statement, and it is why roughly half of the engineering effort in this project went into the viewer and the deployment rather than into the model.

1.6 Expected deliverables

“Deliver a fully integrated software suite with complete source code and technical documentation. The solution must be deployable as a unified module containing the following components:”

- **Elevation Estimation Module:** accept single-view optical satellite imagery in PNG, JPG, or TIFF format and output a high-fidelity DSM in a standard geospatial format.
- **Interactive Visualization Platform:** provide a user-friendly 3D flythrough experience that lets users upload imagery, visualize reconstructed terrain, and validate estimated height values against reference datasets.

1.7 The dataset field, and the fact that it is truncated

The portal prints, and cuts off mid-word:

```
Any high-resolution remote-sensing dataset openly available on the internet may be used
for development. Reference dataset: https://github.com/IMG-PROCESS-SAC/SIH2026/. A
lower-resolution DEM source such as SRTM 30 m may be used to map scale-agnostic depth
features to absolute metric elevations. During final evaluation, ISRO RGB-band optical
satellite i
```

The visible direction is that **final evaluation uses ISRO RGB-band optical satellite imagery** — a different sensor and a different ground resolution from any training set available to us. Cross-sensor generalisation is therefore a scored risk, not a nicety. This is the reason the `--auto-zoom` resolution-matching path exists (Part 8.5).

1.8 What ISRO's own repository adds

`IMG-PROCESS-SAC/SIH2026` redirects to `IMG-PROCESS-SAC/SIH-DepthWizard-2026` . Its README restates the portal text and adds a dataset recommendation:

GAMUS	Provider Earthflow, CC-BY-4.0, described as the “recommended open-source foundation.”
-------	---

Structure	images/ , heights/ (as *_AGL.h5), classes/
Size	5,004 train / 859 val / 2,861 test = 8,724 tiles
Cities	Philadelphia 4,398 · New York 2,167 · Washington DC 2,159 — all dense US urban
Dataloader	github.com/EarthNets/RSI-MMSegmentation
Escape hatch	“While GAMUS is recommended, you are free to utilize any open-source dataset containing remote-sensing depth data, provided it supports both relative depth training and metric scale calibration.”
Stated purpose	“Use this data to overcome the domain gap between natural egocentric imagery... and top-down remote sensing imagery.”

1.9 Five deltas between the portal and what the team had assumed

1. **Deadline is 30 September, not 20 September.**
2. **GAMUS is the sponsor-recommended dataset; this project was built on DFC2019.** DFC2019 remains permitted — the escape hatch above says so explicitly — but GAMUS is three times the tiles and carries the sponsor's endorsement. Integration status is in Part 6.6.
3. **Structural height and slope analysis is required, not a differentiator.** “analysis of structural heights and slopes from arbitrary aerial perspectives” sits in the Description. The measurement tool is therefore a requirement, not a bonus.
4. **“Standalone application” / “standalone deployment” appears three times**, including inside the 50% rendering score. This is why `viewer_standalone.html` exists as a single double-clickable file.
5. **The theme is Disaster Management**, not Space Technology.

What the system does, end to end

One optical satellite image goes in. A navigable, measurable 3D surface comes out. This part traces every stage of that path and names the file that performs it.

2.1 The one-sentence description

THE CLAIM, STATED PRECISELY

DepthWizard takes a **single** optical satellite photograph and produces a per-pixel map of height above ground in **metres**, together with a per-pixel statement of how uncertain each of those metres is, then renders that map as a textured 3D surface a user can fly through and take measurements on in a web browser — with no graphics card anywhere in the chain.

2.2 Why one image is hard

Height from imagery is normally obtained one of three ways, and all three need something we are refusing to require:

CONVENTIONAL METHOD	WHAT IT NEEDS	WHY THAT IS A PROBLEM
Stereo photogrammetry	Two photographs of the same ground from different angles	Cannot be applied retrospectively. Decades of single-view archive imagery can never gain height.
Airborne LiDAR	A laser scanner flown over the area	Expensive, slow, requires a flight campaign per area.
InSAR	Radar satellite pairs with controlled baselines	Specific sensor availability; computationally heavy.

Single-view height estimation is **ill-posed**: infinitely many three-dimensional scenes project to the same flat photograph. There is no unique correct answer. That is not a detail — it is the reason this project predicts a *distribution* per pixel rather than a number, which is covered in Part 5.

2.3 The full pipeline

RAW ARCHIVE
|
v
SHARDED DATA
|
v
CHECKPOINT
|
v
RASTER PRODUCTS
|
v
BROWSER BUNDLE
|
v
3D FLYTHROUGH

DFC2019 zip archives (RGB + LiDAR truth + semantic classes)
tools/prepare_data.py extract -> probe -> shard

data/shards/*.npz 518x518 crops, region-disjoint split
train.py DA-V2 backbone + twin (mu, log-var) head

checkpoints/run02/best.pt 297.1 MB (99.2 MB stripped for serving)
infer.py sliding window 518 / overlap 140, cosine blend

scene.height.tif scene.sigma.tif float32 GeoTIFF, metres
tools/export_terrain.py

viewer/scenes/<name>/ height.bin sigma.bin truth.bin texture.jpg
 + manifest.json

viewer/index.html + main.js + three.js -> WebGL mesh, measurable

2.4 Stage by stage, with the responsible file

1. Ingest

tools/prepare_data.py unpacks the DFC2019 archives, *measures* what the documentation does not state (pixel size, void encoding, height range), and writes compressed shards. The probe step exists because the DFC2019 tiles were rewritten by **tiffiffle** at some point in their history and lost all georeferencing — nothing in the files says a pixel is 30 cm.

2. Split	Regions are assigned whole to train, validation or test with seed 1337. No tile from a training region can appear in validation. 76 / 16 / 16 regions.
3. Train	<code>train.py</code> fine-tunes Depth Anything V2 Small with a Gaussian negative-log-likelihood objective, two learning rates, and a software thermal governor for the GPU.
4. Score	<code>tools/evaluate.py</code> runs whole 1024×1024 tiles through a sliding window and reports RMSE, MAE, correlation, bias, per-semantic-class error, per-terrain error, per-building error, calibration error, and the σ -versus-error rank correlation.
5. Infer	<code>infer.py</code> runs a scene of any size: tiles it at 518×518 with 140 px overlap, blends with a cosine window, optionally applies 8-fold test-time augmentation, optionally fuses a second pass at 2× zoom, optionally matches the input resolution to the training resolution, optionally anchors to a coarse DEM or to ground control points. Writes GeoTIFFs.
6. Export	<code>tools/export_terrain.py</code> converts the rasters into a browser bundle: raw float32 binaries plus a JSON manifest carrying provenance, geometry, and every statistic the viewer's panels display.
7. Render	<code>viewer/main.js</code> builds a Three.js mesh from the float32 heights, drapes the original photograph over it, and provides four display modes, a drag-to-compare against LiDAR, an error map, a sun-direction control, a tour path, and a two-click measuring tool with calibrated error bars.
8. Serve	<code>tools/serve_app.py</code> exposes the whole thing publicly: the viewer, the baked scenes, and an upload endpoint that runs stages 5–6 on demand.
9. Package	<code>tools/build_standalone.py</code> inlines everything — Three.js, the application code, every scene binary, every texture — into a single 15.16 MB HTML file that runs offline from a double-click.

2.5 The two output products the problem statement asks for

INPUT	PRODUCT	HOW IT IS PRODUCED
PNG / JPG, no spatial metadata	rDSM — relative surface model	The model's height output is used directly. It is metric in the sense that the head was trained to regress metres, but with no location there is no datum to anchor it to, so it is reported as relative.
GeoTIFF with a CRS and transform	Absolute DSM — metric elevation	DSM = terrain elevation + above-ground height . The terrain comes from Copernicus GLO-30 fetched for the scene's bounds; the above-ground height is ours. Enabled with <code>--dem auto</code> .

THE DISTINCTION THAT MOST CONFUSES READERS

The model predicts **AGL** — above ground level, also called nDSM or normalised DSM. A 10 m building on a 100 m hill reads 10 m, not 110 m. That is deliberate: it is the quantity that transfers between places, because it does not depend on where sea level is. A **DSM** is elevation above a vertical datum, so producing one means adding the terrain back. Part 8.6 covers how.

2.6 What makes this more than a depth model with a camera flying through it

- Every pixel carries an error bar.** The network predicts a Gaussian per pixel, trained with heteroscedastic negative log-likelihood. The claim is falsifiable and was tested: expected calibration error is 0.077 and the rank correlation between predicted uncertainty and actual error is +0.829.
- The viewer is a measuring instrument.** Two clicks give ground distance, height difference and slope distance in metres, with the uncertainty on the height difference propagated — and *calibrated*, because the naive combination of two error bars is wrong by a factor of about 3.5 on a single rooftop.
- Heights survive the trip to the browser exactly.** Terrain ships as raw float32, not as a 16-bit PNG. Browsers decode images through an 8-bit canvas and would silently quantise the metres.
- It runs with no GPU.** ONNX export, int8 quantisation, CPU inference measured at 509 ms per 518×518 tile, and a live public deployment on a 2-vCPU virtual machine.

The complete technology stack

Every language, library, service and format used anywhere in the project, with the version constraint that is actually declared and the reason it is there.

3.1 Languages and runtimes

TECHNOLOGY	VERSION	WHERE IT IS USED
Python	3.10.7	Everything server-side: data preparation, training, inference, evaluation, export, the HTTP server, the deck generators. 19,713 lines across 62 modules.
JavaScript (ES modules)	ES2020, no transpiler	The 3D viewer. <code>viewer/main.js</code> is 43,635 bytes of hand-written code with no framework, no build step and no bundler.
HTML / CSS	—	<code>viewer/index.html</code> , 12,713 bytes. Hand-written, no CSS framework.
Batch script	—	<code>run_viewer.bat</code> — starts a local HTTP server, because the module viewer cannot load from <code>file://</code> .

PYTHON 3.10, NOT 3.11

An earlier version of the pitch deck said 3.11. The interpreter in `D:\sih2026\.venv` is **3.10.7**. This was corrected in the deck on 6 Sep 2026.

3.2 Machine learning stack

LIBRARY	DECLARED VERSION	WHAT IT DOES HERE
PyTorch	declared <code>≥2.4</code> for training, pinned <code>==2.11.0+cpu</code> for serving. Installed locally: <code>2.11.0+cu128</code>	The whole training and inference stack. The serving pin matches the version that wrote the checkpoint. The CUDA build must match the driver: installed from the <code>cu128</code> index for the local RTX 3060.
torchvision	<code>≥0.19</code>	Image transforms during training. Deliberately excluded from the serving requirements — nothing at inference time needs it.
transformers (Hugging Face)	declared <code>≥4.44, <5</code> in <code>requirements-serve.txt</code> . Installed locally: <code>5.16.1</code> — a real inconsistency: the serving pin would install a <code>4.x</code> that differs from the version the checkpoint was developed against.	<code>AutoModelForDepthEstimation.from_pretrained</code> constructs the Depth Anything V2 backbone, neck and head. This is the single call that pulls in the pretrained weights.
huggingface-hub	<code>≥0.24</code>	Model download and local cache management. In the deployment the cache is <i>baked in</i> and <code>HF_HUB_OFFLINE=1</code> is set, so a cache miss becomes a loud failure instead of a slow network call inside a request.
timm	<code>≥1.0</code>	Model-zoo utilities used during backbone probing. Not needed at serving time and excluded from the serving requirements.
ONNX Runtime	<code>onnxruntime</code> , CPU execution provider	Runs the exported model with no PyTorch installed. This is what makes the “standalone deployable module” claim true rather than aspirational.

3.3 Geospatial stack

LIBRARY	VERSION	WHAT IT DOES HERE
---------	---------	-------------------

rasterio	≥1.3	All GeoTIFF reading and writing, CRS handling, affine transforms, windowed reads. The manylinux wheel bundles GDAL, which is why the deployment needs no separate GDAL install.
GDAL	bundled inside the rasterio wheel	The underlying raster driver library. Used through rasterio, never directly.
pyproj	≥3.6	Coordinate reference system transforms — converting scene bounds to latitude/longitude to work out which Copernicus DEM tiles to fetch.
/vsicurl/	GDAL virtual filesystem	Reads Cloud-Optimised GeoTIFFs directly over HTTPS without downloading the whole file. Used for Copernicus GLO-30 and Google Open Buildings.

3.4 Numerical and scientific stack

LIBRARY	VERSION	WHAT IT DOES HERE
NumPy	≥1.24, pinned <2 for serving	Every array operation outside the network itself. The <2 pin on the server avoids the NumPy 2 ABI break against the pinned torch build.
SciPy	≥1.11	Specifically <code>scipy.ndimage : gaussian_filter</code> for the zoom-fusion Laplacian split, <code>percentile_filter</code> for the bare-earth DEM approximation, and connected-component labelling for per-building statistics.
Pillow	≥10.0	Reading and writing PNG/JPG inputs and writing the texture JPEG for each scene bundle.
pandas	≥2.0	Analysis and ablation tables only. Not used at inference or serving time.
matplotlib	≥3.8	Every figure in the evidence pack, generated by <code>tools/make_figures.py</code> . Not used at serving time.
PyYAML, tqdm	≥6.0, ≥4.66	Configuration parsing and progress bars.
PyMuPDF (fitz)	in the venv	Reading research paper PDFs during literature work, and post-processing generated PDFs.

3.5 Front-end and 3D stack

TECHNOLOGY	VERSION	WHAT IT DOES HERE
Three.js	r169, MIT licence	The WebGL renderer. Vendored locally at <code>viewer/vendor/three.module.js</code> (687,458 bytes) so the viewer works offline. The problem statement names Three.js as an acceptable engine.
WebGL	via Three.js	Hardware-accelerated rendering in the browser. Verified working under SwiftShader software rendering, so a machine with a real GPU is strictly more capable than what the test passed on.
Float32Array / fetch	native browser APIs	Heights are transferred as raw float32 binaries and read with <code>Float32Array</code> . No image codec touches the height data, so no quantisation occurs.
No framework	—	There is no React, no Vue, no Svelte, no bundler and no npm dependency tree in this project. The viewer is plain ES modules.

TWO TECHNOLOGIES THAT WERE LISTED IN AN EARLY DECK AND ARE NOT USED

React appears nowhere in the application. It occurred only as the string name of an icon inside the PowerPoint-generation scripts.

OpenCV appears nowhere in the codebase at all. Both were removed from the deck on 6 Sep 2026.

3.6 Infrastructure and hosting

SERVICE	DETAIL	ROLE

Azure App Service	Linux, plan <code>dwz-plan</code> , SKU B2 (2 vCPU, 3.5 GB RAM), region Central India, resource group <code>sih2026-depthwizard</code> , app <code>depthwizard-sih2026</code>	Hosts the entire public product. CPU only — there is no GPU anywhere in the deployment.
Oryx	Azure's Python build system	Installs dependencies on the platform from <code>requirements.txt</code> , so nothing local has to build or push a container image.
Cloudflare DNS	zone <code>zaidansari.tech</code>	CNAME <code>project5</code> → <code>depthwizard-sih2026.azurewebsites.net</code> , plus a TXT record <code>asuid.project5</code> holding the domain verification ID.
Azure managed certificate	free tier, SNI binding	TLS for <code>project5.zaidansari.tech</code> .
Python <code>http.server</code>	standard library <code>ThreadingHTTPServer</code>	The web server itself. No Flask, no FastAPI, no Django, no gunicorn. 506 lines in <code>tools/serve_app.py</code> .
Docker	Dockerfile written and kept working, not used	The container path is correct and is the route back to Azure Container Apps on any subscription that permits it. It is unused here for two measured reasons given in Part 11.
Kaggle	30 GPU-hours per week, 12-hour sessions	Parallel experiments. T4 and P100 GPUs, neither of which has native bf16 — the training code detects this and falls back to fp16.
Git	local repository	Version control. Data, weights and the virtual environment live outside the repository.

3.7 File formats used

GeoTIFF (.tif)	Input imagery and all raster outputs. Carries CRS and affine transform when present. Output height and sigma rasters are float32, single band, metres.
PNG / JPG	Accepted as input for the non-georeferenced path. JPEG is also the scene texture format in the browser bundle.
.npz	Compressed NumPy archives. One shard holds many 518×518 crops with their heights and semantic classes. 92 files: 61 train, 14 val, 14 test, plus three JSON metadata files.
.pt	PyTorch checkpoint. <code>best.pt</code> is 297,134,061 bytes with optimiser, scheduler and RNG state; stripped to 99.2 MB for serving.
.onnx	Exported model. fp32 is 100,672,732 bytes; int8 is 36,814,825 bytes.
.bin	Raw little-endian float32 arrays for the browser: <code>height.bin</code> , <code>sigma.bin</code> , <code>truth.bin</code> , <code>terrain.bin</code> . A 1024×1024 scene layer is exactly 4,194,304 bytes.
.h5	HDF5. The GAMUS dataset stores its height labels as <code>*_AGL.h5</code> .
.IMD	WorldView image metadata. 67 of them carry solar and viewing geometry for the DFC2019 scenes; parsed by <code>tools/parse_imd.py</code> .
JSONL	Append-only training log, one JSON object per line, so a run that dies mid-epoch still plots.

3.8 Licensing — what can and cannot ship

COMPONENT	LICENCE	CONSEQUENCE
Depth Anything V2 Small	Apache-2.0	Can be shipped in a deliverable. This is what we use.
Depth Anything V2 Base / Large / Giant	CC-BY-NC-4.0	Cannot go into a deliverable. Non-commercial only.
Three.js	MIT	Vendored and shipped.

DFC2019 (IEEE GRSS)	free account required, not redistributable	Not committed to the repository. Obtained from IEEE DataPort.
GAMUS	CC-BY-4.0	Usable with attribution.
Copernicus GLO-30	open, no account or key	Fetches at runtime from a public S3 bucket.
CartoDEM (ISRO)	free to Indian Government Entities only	Not used. As students we do not qualify, so the honest choice is Copernicus.
FABDEM (true bare earth)	non-commercial	Not used. Its absence is why the bare-earth correction is an approximation and is labelled as one.
Maxar Open Data	CC-BY-4.0	Source of the Indian imagery at 0.31 m.
pubgeo/dfc2019 baselines	MIT	Reference metrics implementation.

The model

Depth Anything V2 Small, adapted to predict absolute height in metres with a per-pixel uncertainty. Every layer, every parameter count, and every initialisation decision.

4.1 What the base model is

Identifier	depth-anything/Depth-Anything-V2-Small-hf on Hugging Face
Licence	Apache-2.0. This is the only variant that can legally ship. Base, Large and Giant are CC-BY-NC-4.0.
Task it was trained for	Monocular relative depth estimation on natural, ground-level photographs.
Backbone	DINOv2 ViT-S — a vision transformer, patch size 14, embedding dimension 384.
Neck	DPT — Dense Prediction Transformer. Reassembles the coarse token grid back into a dense per-pixel feature map.
Constructed by	One call: <code>AutoModelForDepthEstimation.from_pretrained(model_id)</code> . The backbone, neck and head are then taken apart and rewired.

What each of those words means

Vision transformer	A network that cuts an image into a grid of small squares (“patches”), turns each into a vector (“token”), and lets every token exchange information with every other token. Unlike a convolutional network it has a global view from the first layer.
Patch size 14	Each square is 14×14 pixels. This single number constrains everything downstream — see the input-size trap in 4.5.
Embedding dimension 384	Each 14×14 patch is represented by a vector of 384 numbers.
DINOv2	A backbone trained with self-supervision — no human labels at all. It learned general visual structure from a very large image collection, which is why it transfers to imagery it never saw, including satellite photographs.
DPT	The decoder design that takes transformer tokens at several depths and progressively fuses and upsamples them into a full-resolution prediction.

4.2 Exact parameter counts

Measured on 6 September 2026 by constructing the model and summing `numel()` over each submodule.

COMPONENT	PARAMETERS	SHARE	WHAT IT DOES
Backbone (DINOv2 ViT-S)	22,056,576	89.0%	Understands the image
Neck (DPT)	2,700,768	10.9%	Turns tokens back into dense features
Head (all four convolutions)	27,778	0.1%	Turns features into metres and an error bar
Total	24,785,122 (24.79 M)	100%	

THE SENTENCE THIS TABLE IS FOR

Almost all of the learned knowledge — 89% of it — lives in a backbone that was downloaded, not trained here. The part that was *designed* for this problem is 0.1% of the parameters. That is not a weakness; it is the entire point of fine-tuning a foundation model. But it means the honest claim is “we adapted a pre-trained backbone,” never “we built a 25-million-parameter network.”

The head, layer by layer

LAYER	SHAPE	PARAMETERS	ORIGIN
conv1	Conv2d 64 → 32, kernel 3×3	18,464	Pretrained, kept
conv2	Conv2d 32 → 32, kernel 3×3	9,248	Pretrained, kept
conv_mu	Conv2d 32 → 1, kernel 1×1	33	Pretrained conv3 , repurposed as the height branch
conv_log_var	Conv2d 32 → 1, kernel 1×1	33	New. The uncertainty branch. Added by this project.

The uncertainty head is **33 parameters**: 32 weights and one bias. That is the entire cost of the project's headline differentiator.

4.3 The four changes made to the stock model

Change 1 — two outputs instead of one

The stock head ends in a single 32→1 convolution. That convolution is kept and becomes the **mean branch** (conv_mu), because its weights are pretrained and worth keeping. A second 1×1 convolution is added as the **log-variance branch** (conv_log_var). Both share conv1 and conv2 .

This is the standard **Kendall & Gal arrangement** for heteroscedastic uncertainty: one trunk, two thin heads, one predicting the value and one predicting how uncertain that value is. It costs almost nothing.

Change 2 — no final ReLU

The stock head clamps its output at zero, because relative depth cannot be negative. Height above ground is *almost* non-negative — but a hard ReLU has zero gradient everywhere it is active, so any pixel the model pushes below zero early in training is dead for the rest of the run. The ReLU is removed and the loss is left to handle plausibility.

Change 3 — output scaled to metres

The pretrained head emits values around 1. Height above ground runs to tens or hundreds of metres. Rather than force the head to learn a large gain through a 5×10⁻⁶ learning rate, the network predicts in normalised units and the result is multiplied by height_scale .

HEIGHT_SCALE = 13.223477220535276

This is not a round number and not a guess. It is the **95th percentile of the above-ground height distribution** measured across all 2,783 training tiles. The maximum would be wrong — that is a lone 204 m structure, and scaling by it would squash every ordinary building toward zero. The p95 puts the bulk of the distribution in a sensible numeric range.

The loss therefore sees metres directly, which means the RMSE printed during training is already the number ISRO scores, while the network's internal range stays around 1.

Variance is predicted in the same normalised units, so converting it to metres is a shift of 2·log(height_scale) in log space. Doing that inside the model rather than inside the loss keeps the loss unit-agnostic and the reported σ in metres.

Change 4 — a useful accident of polarity

Depth Anything V2 predicts *inverse* relative depth: nearer is a larger number. Viewed from orbit, nearer means taller. So the pretrained head's sign convention already matches height, and the fine-tune starts from a sensible initialisation rather than fighting one.

4.4 Initialisation, and why each choice is deliberate

conv_log_var weights	Initialised to zero, bias to 2·log(init_sigma / height_scale) with init_sigma = 5.0 m . So σ starts uniform at 5 m everywhere and only becomes spatially varying once the data gives it a reason to. Random initialisation here lets the variance term shove the mean head around before the mean head knows anything.
conv_mu, default	init_mu="pretrained" — keeps the pretrained readout. This is what runs 01–04 used, and keeping it as the default means those runs stay bit-comparable.

conv_mu, alternative

init_mu="constant" — zero the weights and set the bias to a chosen constant height. Added after run05 died. See below.

THE INITIALISATION BUG THAT COST NINE GPU-HOURS

The pretrained readout emits **disparity**, and its output scale is a property of the checkpoint's feature magnitudes, not of the task. Measured on real crops with `tools/preflight.py` : the median absolute residual at initialisation is **16.5 m for DA-V2-Small** but **666 m for DA-V1-Large**. The two model families' necks do not emit features of the same magnitude, even though their `conv3` weights are comparable in size (mean $|w|$ 0.088 against 0.100).

A 666 m residual drives `log_var` into its clamp on the very first NLL step. The clamp then zeroes the variance head's gradient *and* suppresses the mean head's by roughly 1,100× (measured by `tools/nll_deadlock_probe.py`). That is what killed run05: it diverged to NaN at epoch 6 and produced no information at all. The fix is `init_mu="constant"` , which discards a 32→1 linear readout of disparity that would have to be relearned as height anyway, while keeping the backbone, the neck and conv1/conv2.

4.5 The input-size constraint — a trap that fails silently

PATCH SIZE 14 GOVERNS EVERY IMAGE DIMENSION IN THIS PROJECT

The head computes `patch_h = H // 14` and then interpolates its output to `patch_h × 14` . Feed it a 256×256 crop — the size the Depth Any Canopy recipe uses — and you get **252×252** back, with no warning and no error.

Crops are therefore **518**, which is 14 × 37 and is also DA-V2's native training resolution. A stride of **506** tiles a 1024×1024 image exactly 2×2. `model.check_input_size()` raises a `ValueError` naming the nearest valid sizes rather than letting a mismatch reach the loss, where it would surface as a confusing broadcast error or, worse, silently work after an accidental resize.

4.6 The forward pass, in order

```
1. check_input_size(H, W, 14)           raise if not a multiple of 14
2. ph, pw = H // 14, W // 14           token grid dimensions
3. out = backbone.forward_with_filtered_kwargs(x)  DINOv2 ViT-S -> feature maps
4. hidden = neck(out.feature_maps, ph, pw)  DPT reassemble + fuse
5. feat = hidden[head_in_index]          head_in_index = -1, the last stage
6. x = conv1(feat)                       64 -> 32 channels
7. x = interpolate(x, (ph*14, pw*14), bilinear)  upsample, exactly as in the DPT paper
8. x = activation1(conv2(x))             32 -> 32, ReLU
9a. mu = conv_mu(x) * height_scale       metres
9b. log_var = conv_log_var(x) + 2*log(height_scale)  log-variance in metres-squared
10. return mu, log_var
```

`model.predict()` wraps this, converts log-variance to `sigma = exp(0.5 · log_var)` , and returns `(height_m, sigma_m)` as two `(B, 1, H, W)` tensors.

4.7 The optional binned head (built, measured, not shipped)

A second head design exists behind `--bins N` , defaulting to `0` (off). It implements the architecture from HTC-DC Net, IEEE TGRS 2023:

Adaptive bins	A small MLP over the pooled feature map predicts N bin <i>widths</i> per image via softmax, so widths are strictly positive and always sum to the full height range. Edges are the cumulative sum from <code>bin_min</code> .
Soft-argmax	The output is the expectation $H = \sum P_i c_i$ over bin centres — a continuous value from a classification head, with no discretisation artefacts.
Head-tail cut	Foreground is defined as height > 1 m. Foreground and background get their <i>own</i> distribution over bins, selected per pixel by a binary classifier. Measured on our shards, 35.6% of valid pixels sit above 1 m, so the binary problem is close to balanced.
Warmup routing	During the first 500 steps the ground truth does the routing, not the gate. Measured at initialisation, a randomly initialised gate fired on either 0.0% or 99.4% of pixels depending on the random projection — whichever branch loses gets no gradient at all. Truth-routing removes that lottery.

Configuration

N = 128 bins over a measured range of $[-3, 120]$ m. 128 rather than the conventional 256 because ablations show gains saturate early and 128 is what fits in 12 GB at 518×518 .

It lost. run03 scored 6.950 m whole-tile against run02's 6.454 m — 7.7% worse overall and 8.8% worse on buildings, which is the thing it was built to fix. The full result is in Part 13. The code is kept because it exports to ONNX cleanly and is the right starting point if the approach is revisited.

4.8 Parameter groups and learning rates

`model.param_groups()` returns four groups:

GROUP	LEARNING RATE	WEIGHT DECAY
backbone, weights	5e-6	0.01
backbone, biases and norms	5e-6	0.0
head (neck + conv1 + conv2 + conv_mu + conv_log_var), weights	2.5e-4 (50×)	0.01
head, biases and norms	2.5e-4	0.0

The backbone is a well-conditioned pretrained representation and wants the small rate the Depth Any Canopy recipe uses. The head is doing something the pretrained weights were never asked to do — emit metres, and emit a variance from a freshly initialised convolution — so it needs a materially larger rate or it spends the whole budget catching up. Excluding biases and normalisation parameters from weight decay is standard practice for vision transformers.

4.9 Reconstructing a model from a checkpoint

`model.config()` returns everything needed to rebuild: `model_id`, `height_scale`, `predict_uncertainty`, `bins`, `bin_min`, `bin_max`, `htc`. It exists in one place because three separate tools reconstruct a model from a checkpoint, and each used to hardcode the keys it knew about — adding the binned head would have broken all three silently. `from_checkpoint()` defaults `htc` to `False` for older checkpoints, because any checkpoint written before the head-tail cut existed was trained without it.

The loss functions

Why the model predicts a distribution rather than a number, what each loss term buys, and the tests that prove the uncertainty head recovers a known answer.

5.1 The core objective — heteroscedastic Gaussian negative log-likelihood

$$L = 0.5 * (\log_var + (y - \mu)^2 / \exp(\log_var))$$

Read the two terms as a trade. The model can lower the residual term by admitting uncertainty — but it pays `log_var` for doing so. It cannot buy a free pass by predicting huge variance everywhere, because the first term grows.

Heteroscedastic means the uncertainty is predicted per pixel rather than being one number for the whole image. That is the useful version: a flat car park and a cluttered rooftop edge genuinely do not deserve the same error bar.

Predicting *log*-variance rather than variance keeps it positive without a clamp and keeps gradients well-scaled.

5.2 Every term in the composite loss

TERM	DEFAULT WEIGHT	WHAT IT DOES
Gaussian NLL	1.0	The main objective. Fits the height and its uncertainty jointly.
Gradient matching	0.5 (<code>--grad-weight</code>)	Matches the spatial gradient of the prediction to the gradient of the truth. Sharpens edges; without it surfaces read as rounded mounds.
MSE warm-up	first 200 steps	Plain mean-squared error for the first 200 steps so <code>mu</code> becomes reasonable before the variance head starts shaping gradients.
β -NLL	0.5 in run02 (<code>--beta</code>)	Detach-weighting of the residual term, per Seitzer et al. 0 is standard NLL. Small positive values stabilise very early training.
Chamfer (binned head only)	0.01	Pulls the bin <i>edges</i> onto the quantiles of the truth — the edges, not the centres, which is what HTC-DC Net supervises.
Distribution constraint (binned only)	1.0	Cross-entropy against a Gaussian centred on the truth, supervising the <i>shape</i> of the per-pixel distribution rather than only its mean.
Head-tail cross-entropy (binned only)	1.0	Trains the foreground/background gate directly against (truth > 1 m).

5.3 Clamps and guards

log_var range	Clamped to <code>[-7, +7]</code> (<code>--log-var-max 7.0</code>). Guards against <code>exp()</code> overflow and against the model escaping into “infinitely uncertain everywhere.”
Validity mask	Every loss takes a mask. LiDAR void pixels and no-data regions contribute nothing.
Gradient clipping	Norm clipped at 1.0 (<code>--clip</code>).
Rail tripwire	If <code>log_var</code> sits at its clamp for <code>--rail-patience 50</code> consecutive steps, the run aborts. This is the guard that run05 did not have.
Sanity gates	A run is killed if RMSE exceeds <code>--gate-rmse 25.0</code> or correlation falls below <code>--gate-corr 0.20</code> . <code>tools/tripwire_selftest.py</code> proves both that the tripwires fire on a broken run and that they do <i>not</i> fire on a healthy one.

5.4 The test that proves the uncertainty head works

RECOVERY OF A KNOWN HETEROSCEDASTIC SIGMA

A synthetic dataset is built with two regions of deliberately different noise: true $\sigma = 0.50$ in one and 3.00 in the other. The head is trained on it and asked what σ it thinks each region has. It recovers **0.497** and **2.99**, and separates the two regimes with a ratio of 6.02 against a true ratio of 6.0.

This matters because “the model reports uncertainty” is a claim that is easy to make and hard to check on real data, where the true uncertainty is unknown. On synthetic data the true answer *is* known, and the head returns it.

Test suite status, run on 6 September 2026: `tests/test_losses.py` reports **19 passed, 0 failed**. `tests/test_gcp_affine.py` reports **all 9 tests passed**, including exact recovery of a known ratio error — a fitted exponent of 1.3889 against a true 1.3889, with a residual root-mean-square of 7.74×10^{-7} m.

Both files must be run with `PYTHONPATH` set to the repository root; there is no `pytest` in the virtual environment, and each file carries its own `__main__` runner.

5.5 Why uncertainty is a differentiator and not decoration

Single-view height is ill-posed. A model that outputs one confident number per pixel is hiding that fact. Three consequences make the uncertainty head worth its 33 parameters:

- **It scores in both halves of the marking scheme.** It is an accuracy claim (“calibrated confidence”) and a visualisation feature (a display mode in the viewer).
- **It is operationally usable.** The rank correlation between predicted σ and actual error is **+0.829**. Where the model says it is unsure, it *is* wrong, monotonically. That makes the uncertainty layer usable for triage — deciding which parts of an output need a human to look at them.
- **It feeds the measurement tool.** A height difference between two clicked points carries an error bar derived from the two σ values, which is what turns a picture into an instrument.

The data

Every dataset touched by this project, what is in it, how it was split, and the statistics that were measured rather than assumed.

6.1 DFC2019 Track 1 — the training corpus

Full name	IEEE GRSS Data Fusion Contest 2019, Track 1, also published as the US3D dataset
Source	ieee-dataport.org/open-access/data-fusion-contest-2019-dfc2019 — free account required
Coverage	Jacksonville, Florida and Omaha, Nebraska. Roughly 100 km² in total. Both are flat .
Imagery	WorldView-3 satellite, pan-sharpened RGB, 0.3 m ground sample distance
Reference	Airborne LiDAR, converted to above-ground level. This is the same source the field benchmarks against.
Semantic classes	Ground, vegetation, building, water, bridge
Tile shape	1024 × 1024 × 3 RGB, paired with a 1024 × 1024 float AGL raster
Volume	2,783 tiles across 108 regions
Region naming	JAX_nnn and OMA_nnn . 53 Jacksonville regions, 55 Omaha regions.

THE DFC2019 TILES CARRY NO GEOREFERENCING AT ALL

Measured, not assumed: the probe reports `crs = ['None']` and a pixel size of `1.0` — the identity transform. The tiles were rewritten by `tiff file` at some point in their history, which stripped the CRS and the affine transform. **Nothing in the files says a pixel is 30 cm.**

Two consequences run through the whole project. Ground sample distance must be passed explicitly with `--gsd 0.3` . And `tools/export_terrain.py` refuses to invent one: with no GSD it labels distances in *pixels* rather than printing confident nonsense in metres.

Measured height distribution across all 2,783 tiles

STATISTIC	VALUE	WHAT IT TELLS YOU
Minimum	-2.976 m	Slightly negative — excavations, and LiDAR noise around the zero plane
1st percentile	-0.077 m	
25th percentile	0.000 m	A quarter of all pixels are exactly ground
Median	0.037 m	Half the scene is flat ground. This is the long tail that makes the problem hard.
75th percentile	4.594 m	
90th percentile	9.853 m	Used to initialise the foreground bin head
95th percentile	13.223 m	This is height_scale
99th percentile	21.063 m	
99.9th percentile	44.266 m	
Maximum	204.147 m	One structure. Scaling by this would squash everything else to nothing.

Void pixels are encoded as a sentinel, not as NaN (`void_is_nan = False`). This is the kind of thing that has to be measured: assuming NaN would have let void pixels enter the loss as real zero-height ground.

6.2 The split — region-disjoint, seed 1337

SPLIT	REGIONS	SHARD FILES	ROLE
train	76	61 × .npz	Fine-tuning
val	16	14 × .npz	Every development decision and every reported number
test	16	14 × .npz	Held out. Spent twice by accident before the <code>--split</code> flag existed; not scored since.

WHY REGION-DISJOINT, WHEN IT MAKES OUR NUMBERS LOOK WORSE

Whole regions go to one split. No tile from a training region can appear in validation.

A random tile split leaks: adjacent tiles share buildings, lighting, architecture and shadows, so a model can score well by memorising a neighbourhood. The published HTC-DC Net DFC19 results randomly split 44,258 crops of 256×256 from two cities, which means the crop immediately next to a test crop — same building, same rooftop, same street — is in the training set.

This is why our 4.68 m zero-shot figure and their 2.12 m are not the same number. Presenting our result as losing to theirs would be wrong, and quietly adopting their protocol to close the gap would be worse.

THE TEST SPLIT IS MATERIALLY EASIER THAN VALIDATION

The same zero-shot global-affine baseline reads **4.68 m on test** against **9.308 m on validation**. The test split also contains **exactly one building above 30 m** (maximum 34.0 m), while validation reaches 155.3 m. When a test number is finally reported it will look better than validation, and that must be said whenever it is quoted.

6.3 The validation set used for every reported number

Tiles	80 whole 1024×1024 tiles, drawn from the 16 validation regions with seed 1337
Valid pixels	11,983,760
Buildings	3,090, after dropping connected components smaller than 25 m²
Median building height	4.22 m
90th percentile building height	7.53 m
Tallest building	155.3 m

The tile list is bit-identical across every method compared in Part 9 — verified, not assumed.

6.4 The crops — what training actually sees

Crop size	518 × 518 = 14 × 37. DA-V2's native resolution and a multiple of the patch size.
Storage	Compressed NumPy .npz shards. Each holds many crops with their RGB, height and semantic class.
Loader	depthwizard/dataset.py — ShardStream plus HeightShardDataset, with a two-stage shuffle: shards are shuffled, then crops within a buffer of <code>--buffer-shards 2</code> shards are shuffled again. This gives good mixing without holding the corpus in RAM.
Augmentation	D4 — the eight symmetries of a square (four rotations × optional mirror) — plus photometric jitter. D4 is safe here in a way it is not for ground-level photography, because a satellite image has no privileged “up”.
Normalisation	ImageNet mean and standard deviation, matching what the pretrained backbone expects.

6.5 Solar and viewing geometry

`tools/parse_imd.py` parses WorldView `.IMD` metadata into JSON. **67 files** were recovered and joined to Track 1 tiles by scene index. Each carries, for example:

```
{
  "CatId": "104001000209A800",
  "satId": "WV03",
  "firstLineTime": "2014-10-05T16:01:38.873575Z",
  "meanOffNadirViewAngle": 18.9,
  "meanInTrackViewAngle": 18.6,
  "meanCrossTrackViewAngle": 3.6,
  "meanSatAz": 19.8,
  "meanSatEl": 69.3,
  "meanSunAz": 150.3,
  "meanSunEl": 50.8,
  "shadow_len_per_m": 0.8156,
  "shadow_quality": "usable"
}
```

Sun elevation across the set spans **23.4° to 74.5°**, and **54 of the 67** scenes have a usable sun elevation for shadow work. This metadata is what made the shadow investigation in Part 13 possible — and what allowed it to be closed with measurements rather than opinion.

6.6 GAMUS — the sponsor's recommended dataset

PROPERTY	VALUE
Source	<code>huggingface.co/datasets/earthflow/GAMUS</code> — provider Earthflow, CC-BY-4.0
Added to ISRO's repository	29 August 2026
Size	8,724 tiles: 5,004 train / 859 val / 2,861 test
Cities	Philadelphia 4,398 · New York 2,167 · Washington DC 2,159 — all dense US urban
Structure	<code>images/</code> , <code>heights/</code> as <code>*_AGL.h5</code> , <code>classes/</code>
Ground sample distance	0.33 m — within 6% of our 0.3 m, so the resolution match is close
Void encoding	−5 sentinel, not NaN. Measured. Treating it as a real height would poison training.
Disk required	~100 GB against ~31 GB free. This is the blocking constraint.

Why GAMUS matters and why it is not the only corpus

- **It is 36× our tall-building supervision.** Our single measured defect is buildings above 20 m, and the cause is proven data scarcity. Three dense American cities is exactly the missing population.
- **It is the only route to a credible generalisation claim.** Training on two cities and testing on the same two cities cannot demonstrate transfer. Training on DFC2019 and holding out a whole GAMUS city can.
- **It is the sponsor's own recommendation**, which carries weight independent of its technical merits.
- **Not GAMUS alone**, because it is entirely dense urban. Replacing DFC2019 with it would discard our sparse, forested and mixed coverage — three of the four landscapes ISRO explicitly names.

Current status, 6 September 2026

COMPONENT	STATE
<code>tools/prepare_gamus.py</code>	Written. <code>probe</code> run and passing. 200 tiles ingested , producing 7 shard files in <code>data/shards_gamus/</code> . The ingest is resumable and picks up exactly where it stopped.
<code>tools/link_union.py</code>	Written, not yet run . Builds a union directory as hardlinks, costing zero extra bytes.
<code>tools/evaluate.py</code>	<code>--corpus gamus</code> added. The DFC2019 path is unchanged and bit-identical.
Training on GAMUS	Not started. Blocked on disk. No model in this project has seen a GAMUS tile.

The ingest and a training run must not be run at the same time: the machine has 15.9 GB of RAM with roughly 1 GB free, and the ingest has already pushed the page file up once.

6.7 Indian imagery – Maxar Open Data

Source	Maxar Open Data programme, CC-BY-4.0, free 0.3–0.6 m imagery including India
Scene	Teesta valley, Sikkim. Event folder <code>India-Floods-Oct-2023</code> , but the actual acquisitions in it are dated 2022-03-14 — so the matching Open Buildings year is 2022, not 2023.
Resolution	0.31 m, essentially our training resolution
View geometry	26° off-nadir, which means a roof sits about 5 m from its own footprint — a co-registration problem addressed in Part 9.7
Relief	947 m across the 2 km crop, median slope 31.4°. Far outside anything in DFC2019.
Tooling	<code>tools/maxar_opendata.py</code>

Nepal imagery from the same programme was also cropped for upload testing — three 1024×1024 tiles in `D:\sih2026\testkit\` .

6.8 Reference and anchor datasets

DATASET	WHAT IT IS	HOW IT IS USED
Copernicus GLO-30	Free worldwide elevation, 30 m posting, derived from TanDEM-X. Bucket <code>copernicus-dem-30m.s3.amazonaws.com</code> , no account or key.	The metric anchor. Converts our above-ground heights into an absolute DSM. Also supplies the Himalayan terrain the Sikkim scene stands on.
SRTM 30 m	Shuttle Radar Topography Mission, 2000. Named in the problem statement.	Not used. GLO-30 is the same resolution, needs no account, and covers latitudes above 60°N where SRTM has no data.
CartoDEM	ISRO's Indian elevation product	Not used. Free only to Indian Government Entities. As students we do not qualify.
FABDEM	True bare-earth product	Not used. Non-commercial licence. Its absence is why the bare-earth correction is an approximation.
Google Open Buildings 2.5D Temporal	Building heights inferred from Sentinel-2 at 10 m, effective resolution roughly 4 m. Field <code>building_height</code> , defined as height above terrain in [0, 100] m — the same quantity we predict.	The only quantitative cross-check available over India, because there is no Indian LiDAR. Read as a windowed COG by <code>tools/open_buildings.py</code> .
Bhoonidhi (NRSC)	ISRO's own data portal. Client written in <code>tools/bhoonidhi.py</code> — authenticate, list collections, search, download.	Investigated 27 Aug 2026. Resolved, but it did not yield what was hoped for: the openly available products are not at a resolution useful for building heights.

AN HONESTY NOTE THAT BELONGS IN ANY REPORT

GLO-30 is itself a *surface* model, not bare earth, so in dense urban areas it already contains part of the building height we are about to add. At 30 m posting a single building contributes only a fraction of a cell, but the double-count is real and is largest exactly where buildings are tallest and densest. SRTM has the same property. `--dem-bare` applies a coarse low-percentile filter as an approximation and says that it is one.

Training

The recipe, the exact hyperparameters of the shipping model, all six runs with their outcomes, and the thermal problem that governed everything.

7.1 The recipe and where it comes from

Training follows **Depth Any Canopy**, which is the only published adaptation of Depth Anything to aerial height that reports its hyperparameters: AdamW, backbone learning rate 5×10^{-6} , 5% linear warm-up then linear decay, a handful of epochs.

Two deliberate departures: a **Gaussian negative-log-likelihood objective instead of MSE**, and a **separate, larger learning rate for the head**.

7.2 run02 – the shipping model, exact configuration

Read directly out of `checkpoints/run02/best.pt` on 6 September 2026.

ARGUMENT	VALUE	NOTE
model_id	depth-anything/Depth-Anything-V2-Small-hf	Apache-2.0
height_scale	13.223477220535276	p95 of the measured height distribution
epochs	12	Best epoch was the last, epoch 11
batch	8	Measured as the throughput-efficient setting
accum	1	No gradient accumulation
lr	5e-06	Backbone
lr_head	None → 50× backbone = 2.5e-04	
weight_decay	0.01	Biases and norms excluded
init_sigma	5.0 m	Uniform σ at initialisation
grad_weight	0.5	Gradient-matching term
warmup_mse	200 steps	Plain MSE before NLL takes over
beta	0.5	β-NLL. This is what distinguishes run02 from run01.
clip	1.0	Gradient-norm clipping
precision	auto → bf16 on the RTX 3060	Ampere has native bf16
seed	1337	Same seed as the data split
workers	2	
buffer_shards	2	Two-stage shuffle buffer
val_batches	60	In-training validation sample
max_temp / temp_ceiling	83 °C / 87 °C	Software thermal governor
freeze_backbone	False	Everything trains
no_uncertainty	False	The variance head is on
bins	0	Direct regression, not the binned head
Global steps	11,532	

Wall-clock time	7,811 seconds — 2 hours 10 minutes on one RTX 3060
Best crop-wise validation RMSE	7.980 m at epoch 11 (protocol 1 — see Part 9.1)
Checkpoint size	297,134,061 bytes; stripped to 99.2 MB for serving

7.3 In-training validation of run02, by semantic class

Stored inside the checkpoint. This is **protocol 1**, crop-wise, so it is about 1.5 m higher than the whole-tile figure for the same weights.

CLASS	RMSE	MAE	BIAS	PIXELS
Water	1.604	0.614	+0.202	158,807
Ground	2.199	0.797	+0.575	15,929,692
Vegetation	4.270	2.955	-1.406	2,178,882
Bridge	4.640	3.592	-2.416	229,386
Building	21.168	8.602	-6.417	2,750,783
All	7.980	2.066	-0.577	21,465,960

Correlation +0.770, median absolute error 0.220 m, 90th-percentile absolute error 4.668 m, expected calibration error 0.092, σ -error rank correlation +0.84.

7.4 Every training run, and what each one settled

RUN	WHAT IT WAS	EPOCHS	WALL CLOCK	OUTCOME
run01	First fine-tune. Standard NLL, $\beta = 0$.	6	1 h 37 m	Best crop-wise 8.224 at epoch 1. Whole-tile 6.811. Weak uncertainty: ECE 0.163, rank correlation +0.562.
run02	β -NLL at $\beta = 0.5$. Otherwise identical to run01.	12	2 h 10 m	The model of record. Whole-tile 6.454, building 16.37, ECE 0.083, rank correlation +0.836. Everything shipped comes from this checkpoint.
run03	Binned classification head: 128 adaptive bins, soft-argmax, Chamfer on edges, distribution constraint.	8	1 h 57 m	Lost. Whole-tile 6.950 — 7.7% worse overall, 8.8% worse on buildings. Ground improved to 1.75 m, the best of any run, which is the over-smoothing signature.
run04	Binned head <i>plus</i> the head-tail cut.	12	3 h 34 m	Lost. Best crop-wise 7.887 at epoch 5. Later used to test ensembling, which is how we learned the two models make the same mistakes: correlation between their errors is 0.925.
run05	DA-V1-Large backbone, 335 M parameters.	6, then NaN	~9 GPU-hours	VOID. Diverged to NaN from a variance head railed at its clamp. Nine GPU-hours, zero information. Diagnosed afterwards: the V1-Large readout emits a 666 m residual at initialisation.
run06	The experiment run05 was supposed to be, with <code>init_mu="constant"</code> and the rail tripwire.	stopped in epoch 0	933 s	Never finished. The log stops at step 331 with no validation row. There is no checkpoint. run02 remains the shipping model.

THE SHIPPING MODEL IS RUN02, TRAINED ON 27 AUGUST 2026

Everything measured, exported, deployed and demonstrated comes from that single checkpoint. Runs 03 through 06 are recorded results, not candidates.

7.5 Precision selection

`pick_precision()` chooses bf16, fp16 or fp32:

HARDWARE	COMPUTE CAPABILITY	CHOICE
RTX 3060 (Ampere)	8.6	bf16 — native, no loss scaling needed
Tesla T4 (Turing)	7.5	fp16 + GradScaler — Turing has fp16 tensor cores
Tesla P100 (Pascal)	6.0	fp16 + GradScaler — buys memory, not speed; no tensor cores at all
CPU	—	fp32

DO NOT USE `TORCH.CUDA.IS_BF16_SUPPORTED()`

It counts *emulated* bf16 and returns True on hardware that has none. run06's Kaggle log printed “bf16 supported: True” on a Tesla P100, which has no bf16 at all. On a T4 it would likewise pick bf16 and get a slow software path instead of Turing's fp16 tensor cores — the exact opposite of what the function promises. Compute capability is the unambiguous test: bf16 is native from 8.0 onward.

7.6 Throughput, measured

BATCH	MS/STEP	CROPS/S	PEAK VRAM
4	174	23.0	2.21 GB
8	327	24.5	6.01 GB
12	469	25.6	7.57 GB

Throughput plateaus around **25 crops per second**, so batch 8 — the Depth Any Canopy setting — is also the efficient one, and 12 GB of VRAM is not the binding constraint. **Heat is.**

7.7 The thermal problem, and how it was actually solved

Unregulated, the RTX 3060 reached **91 °C in under ten minutes** and down-clocked itself from 1875 MHz to **1492 MHz**. The “full speed” run was not faster, only hotter. `nvidia-smi -pl` is the clean fix but needs an administrator shell, so `train.py` carries a software **ThermalGovernor**: sample every N steps, pause proportional to overshoot, hard-stop above a ceiling until it cools.

THE SOFTWARE GOVERNOR WAS A WORKAROUND FOR A SETTINGS BUG

MSI Afterburner has a **Prioritize** switch between power and temperature, and it defaults to *power* — which makes the temperature limit advisory. The card draws to its power limit and lets temperature land wherever it lands. That is why a limit of 77 °C still produced 91 °C readings, and why raising the fan from 62% to 85% bought only about 3 °C.

	BEFORE	AFTER (PRIORITIZE = TEMPERATURE)
Temperature	91 °C	mean 80.1 °C, peak 85 °C
Clocks	throttled to 1492 MHz	1777–1867 MHz
Throughput	53 steps/min	143 steps/min
12-epoch run	3.6 h	~1.6 h

Cooler *and* 2.7× faster, because the card down-clocks smoothly instead of being stopped and started. Working configuration: fan 85% with auto off, temperature limit 80, power limit 85%, Prioritize = temperature, and the link toggle off so the two limits can be set independently. The software governor now fires zero times and remains only as a backstop.

NEVER CHANGE GPU SETTINGS WHILE A CUDA JOB IS RUNNING

Applying Afterburner settings resets the display driver and invalidates live CUDA contexts. Doing it mid-run killed a training process with `torch.AcceleratorError: CUDA error: unknown error` and silently reverted the power limit to 170 W.

7.8 Built for interruption

Kaggle sessions are capped at 12 hours and can be reclaimed sooner, so everything assumes the process may die at any moment:

- State is checkpointed every epoch and on a wall-clock budget (`--max-hours`).
- `--resume auto` restores model, optimiser, scheduler *and* RNG state.
- The metrics log is append-only JSONL, so a partial run still plots.
- Preprocessing is done locally — it is CPU and disk bound and burns zero GPU quota — then shards are uploaded once as a Kaggle Dataset and every session mounts them instantly.
- Code lives in git and is cloned into notebooks. Code is never written inside a notebook.

7.9 Preflight — refusing to start a broken run

`tools/preflight.py` exists because run05 burned nine GPU-hours producing nothing. It measures the initial residual before a run begins, and it is the tool that revealed the 16.5 m against 666 m difference between DA-V2-Small and DA-V1-Large. A long run that is already broken at step zero is now refused rather than discovered six epochs later.

Inference

How a full scene of arbitrary size is processed, and every optional stage that can be switched on: augmentation, resolution fusion, zoom matching, DEM anchoring and control points.

8.1 Sliding window

The model accepts exactly 518×518. A real scene does not come in that size, so `infer.py` tiles it.

Tile size	518 (<code>--tile</code>), which must be a multiple of 14
Overlap	140 px (<code>--overlap</code>), giving a stride of 378
Blending	A cosine window over the overlap region, so neighbouring windows fade into each other rather than producing visible seams
Variance combination	Variances combine in quadrature , not by averaging the σ values. Averaging standard deviations is wrong and would understate uncertainty in the overlaps.
Batch	4 windows at a time (<code>--batch</code>)

A 1024×1024 tile is covered by a 3×3 grid of windows — nine forward passes.

8.2 Test-time augmentation

`--tta` runs each window **eight times** under the D4 group — four rotations, each with and without a mirror — then un-transforms and averages the results.

CONFIGURATION	WHOLE-TILE RMSE	PER-BUILDING	BIAS	ECE
run02, single pass	6.456	3.771	−0.31	0.083
run02 + TTA	6.369	3.616	−0.34	0.078
shipped: TTA + fused	6.401	3.667	−0.24	0.077

D4 augmentation is legitimate here in a way it would not be for ground-level photography, because a satellite image has no privileged orientation. Rotating it 90° produces another perfectly plausible satellite image.

8.3 Resolution fusion

`--fuse-zoom 2 --fuse-sigma 8` runs a second pass at 2× magnification and fuses it with the first using a **Laplacian split**: the low-frequency component (smoothed with a Gaussian of $\sigma = 8$) comes from the native-resolution pass, and the high-frequency detail comes from the zoomed pass.

A DELIBERATE TRADE, MADE BECAUSE HALF THE MARKS ARE VISUALISATION

Fusion costs **+0.051 m of per-building RMSE — 1.4%** against TTA alone, and buys **+28% edge definition**: buildings read as flat-topped blocks rather than rounded mounds. It also improves bias and calibration slightly.

The fused numbers are the ones reported, because **whatever is shipped must be what is scored**. Reporting the better 3.616 m while shipping the 3.667 m configuration would be dishonest by a margin larger than the difference.

8.4 The two quality presets

PRESET	WHAT RUNS	FORWARD PASSES PER 1024×1024 TILE	CPU TIME

fast (default on the public site)	Single pass, 9 windows	9	5.0 s inference, 13.3 s wall including model load
accurate	×8 D4 TTA + zoom-2 fusion	~180	57.2 s inference, 86.2 s wall

The ratio between them is **11.4×** (57.2 ÷ 5.02), which is the constant the public server uses to scale its size limits between modes. Peak resident memory is 2.05 GB in fast mode and 2.21 GB in accurate mode — both comfortably inside a 3.5 GB host.

8.5 Auto-zoom — surviving a different evaluation resolution

We train at 0.3 m. ISRO will evaluate on their own imagery, likely nearer 0.6–1.0 m. Untreated, that costs a great deal: a backbone token then covers twice the ground, and buildings get averaged with their surroundings.

One token is 14 pixels. At 0.3 m per pixel that is **4.2 m of ground** per token. At 0.6 m it becomes 8.4 m — wide enough to swallow a small house whole.

`--auto-zoom` reads the ground sample distance from the input's geospatial metadata and upsamples so the backbone always sees the scale it was trained at.

INPUT GSD	UNTREATED PER-BUILDING RMSE	WITH --AUTO-ZOOM	RECOVERY
0.3 m (native)	1.350 m	—	—
0.6 m	2.260 m	1.399 m	within 3.6% of never having lost the resolution
1.0 m	3.138 m	1.657 m	roughly half the loss recovered

`--native-gsd` declares what the model was trained at (default 0.3). If the image has no geospatial metadata there is nothing to read, and the flag has no effect — which is correct behaviour, not a silent failure.

8.6 DEM anchoring — producing an absolute DSM

`--dem auto` turns above-ground heights into absolute elevation:

DSM = terrain elevation (Copernicus GLO-30) + our above-ground height

Tile discovery	<code>dem.tiles_for_bounds()</code> returns every 1°×1° GLO-30 tile touching the scene's bounding box. A scene straddling a tile edge is normal, not exotic — assuming one tile is how a coastal or border scene ends up with half its elevation missing and no error raised.
Tile naming	<code>Copernicus_DSM_COG_10_N27_00_E088_00_DEM</code> — constructed from the floor of the latitude and longitude.
Access	Read over HTTPS through GDAL's <code>/vsicurl/</code> , windowed, so only the needed bytes are fetched.
Resampling	Cubic, onto the scene's own pixel grid.
Datum handling	Terrain is stored relative to a datum (<code>terrain_datum_m</code> in the manifest), or the mesh sits 1.3 km from the origin and every camera default aims at empty sky.
Bare-earth option	<code>--dem-bare</code> applies a coarse low-percentile filter to approximate removing GLO-30's own buildings and canopy, and labels the result as an approximation.
Default state	Off in the public deployment. <code>--dem auto</code> reaches Copernicus during a request that already takes a minute, so it is opt-in per deployment rather than a surprise timeout.

8.7 Ground control point calibration

The problem statement names “a limited set of Ground Control Points” as an acceptable route to absolute scale. This is implemented and, importantly, *measured*.

Affine is the wrong functional form

The model's error is a height-dependent **ratio** — 1.20× at 0–3 m, falling to 0.52× above 40 m. No straight line can be above 1 at one end and below it at the other. Measured head to head on held-out buildings, per tile, over the 10 validation tiles containing a building above 20 m:

FORM	CONTROL POINTS (K)	RMSE	VS UNCALIBRATED	MAE	TILES MADE WORSE
uncalibrated	—	7.248 m	—	4.013 m	—
affine	8	6.154 m	−15.1%	4.477 m	8 of 10
power	5	5.730 m	−20.9%	4.202 m	4 of 10
power	8	5.442 m	−24.9%	4.038 m	5 of 10

Affine buys RMSE by spending MAE (+11.6%); the power law takes more RMSE and leaves MAE where it found it (+0.6%). Over all 57 usable tiles the power law is the only form that beats doing nothing at all (−4.4% against affine's +0.1%).

Guards, calibrated against the measurement rather than chosen

- At least 2 control points must lie in the **upper 60%** of their height span. Requiring the upper *half* would refuse 24.6% of the draws that produced the gain; the upper 60% refuses 6.3%.
- The fitted exponent must be ≥ 0.95 , which is what catches “this fit would compress the heights further.”
- Control values are read as a **structure median** — region-grown from the clicked point — not a fixed patch. On one Omaha tile a point sample returned 0.59 m where the truth was 5.6 m.

HONEST END-TO-END STATUS

On the real tiles tested, **the guards decline and the system falls back to offset-only**. On OMA_288_012 the fitted exponent is 0.40–0.57 because that tile under-predicts everything; on JAX_165_018 it is 0.505 because the model is already good there. Both refusals are correct. **It never degrades a result** — it helps only when control points carry building-level heights and the scene genuinely spans a height range.

A selection trap worth remembering

The first probe required 25 buildings per tile and looked clean. It was worthless: those 36 tiles contain **zero** buildings above 20 m (RMSE 1.381 m), while all 60 tall buildings sit in tiles with fewer (RMSE 8.487 m). Dense suburbs have many small buildings and downtowns have few large ones, so a threshold chosen for sample size excluded 100% of the problem.

8.8 Complete inference flag reference

FLAG	DEFAULT	EFFECT
<code>--image</code>	required	Input scene: .tif, .png or .jpg
<code>--out</code>	required	Output prefix; writes <code><prefix>.height.tif</code> , <code>.sigma.tif</code> and <code>.json</code>
<code>--ckpt</code>	none	Checkpoint. Omit for zero-shot DA-V2.
<code>--tile</code>	518	Window size; must be a multiple of 14
<code>--overlap</code>	140	Window overlap in pixels
<code>--batch</code>	4	Windows per forward pass
<code>--tta</code>	off	×8 D4 test-time augmentation
<code>--fuse-zoom</code>	1	Second pass at this magnification, Laplacian-fused

<code>--fuse-sigma</code>	8.0	Gaussian width of the frequency split
<code>--zoom</code>	1	Fixed magnification for a single pass
<code>--auto-zoom</code>	off	Read GSD from metadata and match the training scale
<code>--native-gsd</code>	0.3	The scale the model was trained at
<code>--dem</code>	none	<code>auto</code> for Copernicus, or a path to a DEM raster
<code>--dem-bare</code>	off	Approximate bare-earth correction of the anchor
<code>--gcp</code>	none	JSON file of ground control points
<code>--gcp-form</code>	power	<code>power</code> or <code>affine</code>
<code>--gcp-scale</code>	off	Fit scale as well as offset
<code>--height-scale</code>	from checkpoint	Override metres per normalised unit
<code>--precision</code>	auto	auto / bf16 / fp16 / fp32
<code>--truth</code>	none	An AGL GeoTIFF; if supplied, the result is scored immediately

A KNOWN COSMETIC BUG

`infer.py --help` crashes. A literal `%` in one help string breaks argparse's formatting. It means `--help` is not a way to discover the flags — this table is.

Results

Every measured number, with the protocol that produced it. Mixing protocols is the most common way to say something false about this project, so the protocols come first.

9.1 The four evaluation protocols

#	PROTOCOL	DATA	PRODUCED BY
1	in-training validation	518×518 crops from <code>val_*</code> shards, valid pixels subsampled	<code>train.py</code> → <code>train_log.jsonl</code>
2	whole-tile validation	80 whole 1024×1024 val tiles, sliding window 518 / overlap 140	<code>tools/evaluate.py --split val</code>
3	whole-tile test	the same, on the held-out test regions	<code>tools/evaluate.py --split test</code>
4	India cross-check	Sikkim crops against Google Open Buildings 2.5D	<code>tools/compare_open_buildings.py</code>

PROTOCOLS 1 AND 2 DIFFER BY ABOUT 1.5 M ON THE SAME CHECKPOINT AND THE SAME SPLIT

run02 at epoch 11 reads **7.980 crop-wise** against **6.454 whole-tile**. Overlap averaging in the sliding window plus a different pixel population account for it. Neither is wrong; they are simply not the same measurement.

This was discovered on 27 August 2026 after two numbers from the same checkpoint appeared to contradict each other. **There was no bug**. They were different splits, and nothing in the tooling made that visible at the call site. Both tools now take `--split` , defaulting to val, and scoring test prints a warning.

9.2 How the shipping numbers were measured

```
python tools/evaluate.py --ckpt checkpoints/run02/best.pt --tta \
  --fuse-zoom 2 --fuse-sigma 8 --out out/eval_run02_ship
python tools/make_figures.py --metrics out/eval_run02_ship/metrics.json
```

Data	IEEE GRSS DFC2019 Track 1 (US3D), Jacksonville and Omaha, 0.3 m GSD
Reference	Airborne LiDAR — the same source the field benchmarks against
Split	Region-disjoint, seed 1337
Sample	80 whole tiles, 11,983,760 valid pixels, 3,090 buildings
Configuration	The shipping configuration — TTA plus zoom-2 fusion — not a favourable variant

9.3 Headline — protocol 2

METRIC	VALUE
Per-building RMSE	3.667 m
Per-building MAE	1.498 m
Per-building bias	−0.237 m
Per-building median absolute error	0.894 m
Per-building correlation	+0.778
Whole-tile RMSE	6.401 m
Whole-tile MAE	1.812 m

Whole-tile correlation	+0.803
Median absolute error, all pixels	0.278 m

For scale: the median building in this validation set is **4.22 m** tall and the 90th percentile is 7.53 m.

WHY PER-BUILDING, NOT PER-PIXEL, IS THE HEADLINE

Per-pixel building error is dominated by roof edges, where the prediction crosses from ground to roof and is briefly wrong by the whole building height. It is not comparable to any published figure. The field reports one height per building, so we do too — taking the **median on both sides**. Using a higher percentile of ours against their median manufactures accuracy: measured at **+1.3 m** when that mistake was made.

Components smaller than 25 m² are dropped, which is the convention HTC-DC Net and GlobalBuildingAtlas use.

9.4 The benchmark being beaten

GlobalBuildingAtlas reports **5.9 m** per-building RMSE over Asia. Our 3.667 m is **38% better**, on the same kind of metric.

The comparison is honest in form — both are per-building RMSE against reference heights — but it is not the same geography, and that must be said. It is a benchmark to place our number against, not a head-to-head result.

9.5 By semantic class — protocol 2

CLASS	RMSE	READING
Water	1.324 m	Flat, easy
Ground	1.906 m	Under 2 m. This is what an absolute DSM stands on, and the surface a flood or landslide model integrates over.
Vegetation	4.237 m	A camera cannot see through leaves
Bridge	4.250 m	Thin structures over voids
Building (per pixel)	16.256 m	Roof edges dominate. Not comparable to published figures — use the per-building number instead.

9.6 Where the error actually lives

TRUE HEIGHT	BUILDINGS	RMSE	BIAS
0–3 m	166	1.282 m	+0.49
3–6 m	2,305	1.429 m	+0.29
6–10 m	443	1.947 m	–0.77
10–20 m	116	4.034 m	–1.03
above 20 m	60	23.451 m	–17.16

79% OF THE TOTAL SQUARED ERROR COMES FROM 1.9% OF THE BUILDINGS

Under 10 m — which is 94% of the building stock — the error is 1.4 m. Above 20 m it collapses to 23 m, and the model is on average 17 m too *short*.

Say this before a judge finds it. The direction matters: we under-call, never over-call, so the failure is conservative.

The tail is a data problem, and that was proved rather than assumed. Training data holds 166 buildings above 30 m and tops out at **82.8 m**, while validation reaches **155.3 m**. Fine-tuning hard on the tall buildings we do have moved transfer bias the *wrong* way, from –18.22 to –19.10 m, while the fit improved — textbook overfitting of a tiny sample. More training will not fix this. More tall buildings would.

9.7 Stability across landscapes — the criterion ISRO names

LANDSCAPE	WHOLE-TILE RMSE	MAE	CORRELATION
Sparse	1.837 m	0.655 m	+0.583
Mixed	2.723 m	1.356 m	+0.824
Forested	3.319 m	2.254 m	+0.780
Urban	13.860 m	4.759 m	+0.808

Urban is 4–7× the others, and it is the same story as the tail: cities are where the buildings above 20 m are. **Correlation stays high at +0.808** — the *shape* is right; the *scale* of tall structures is not.

Forested ground error is 2.50 m against sparse's 0.87 m, and that is not a model defect. LiDAR pulses penetrate canopy and measure the actual ground; a camera physically cannot see through leaves. That is optics, not accuracy — and it is why the same model reaches 0.87 m on open ground a few hundred kilometres away.

Hilly is not in this table because DFC2019 has no hills. Jacksonville and Omaha are both flat. That case is covered by Sikkim below, which has no LiDAR, so it is reported as a cross-check rather than as an accuracy figure.

9.8 Does the model know when it is wrong?

METRIC	VALUE	MEANING
Expected calibration error	0.077	When the model says it is 68% confident, it is right about 68% of the time, to within about 8 percentage points
σ -versus-error rank correlation	+0.829	Where the model says it is unsure, it <i>is</i> wrong — monotonically. This is the number that matters operationally.
Mean predicted σ	1.082 m	The typical error bar

The rank correlation is what makes the uncertainty layer usable for triage rather than decoration. It means an operator can sort by confidence and know that the low-confidence regions really are the ones to check.

The measurement tool applies a measured correction on top of this

Naive `hypot( $\sigma_a$ ,  $\sigma_b$ )` assumes the two pixels' errors are independent, and they are not. Measured on 20 validation tiles from run02:

SEPARATION BETWEEN THE TWO POINTS	CORRECTION APPLIED TO THE NAIVE ERROR BAR
0.6 – 3 m	×0.282
3 – 15 m	×0.698
15 – 60 m	×1.163
60 – 300 m	×1.388

Error correlation is +0.94 across a metre and only reaches zero past about 60 m. So the naive error bar is roughly **3.5× too wide** on a single rooftop and **too narrow** across a neighbourhood. `tools/pair_calibration.py` fits this curve per checkpoint; it ships as `viewer/calibration.json`.

9.9 Fine-tuning against doing nothing — protocol 2, identical 80 tiles

METHOD	RMSE	MAE	BIAS	BLDG RMSE	BLDG BIAS	GRND RMSE
Zero-shot DA-V2, raw	9.998	3.856	−2.047	25.38	−13.72	1.85
Zero-shot + global affine (<i>deployable baseline</i>)	9.308	4.528	+0.230	22.47	−9.64	4.17

Zero-shot + oracle affine (<i>not deployable</i>)	7.285	3.147	-0.000	17.33	-2.64	2.99
DepthWizard run01 (ep1)	6.811	2.224	-0.752	16.72	-3.23	2.39
DepthWizard run02 (ep11)	6.454	1.840	-0.569	16.37	-4.70	1.92

Fine-tuning beats the deployable zero-shot baseline by **30.7%** (9.308 → 6.454), and beats the *oracle* affine variant — which is allowed to fit scale and shift from each tile's own ground truth and is therefore not deployable at all.

NEVER QUOTE THE ORACLE AFFINE AS A RESULT

It refits scale and shift using the test tile's own ground truth. It bounds what a purely linear correction could achieve; it is not deployable, because it needs the answer in order to compute the answer.

9.10 India — protocol 4, where there is no LiDAR

No airborne LiDAR reference exists for our Indian scenes, so this is a **cross-check against another model**, not an accuracy measurement. Google Open Buildings 2.5D Temporal is derived from Sentinel-2 at roughly 4 m effective resolution; we run at 0.31 m.

THEIR CONFIDENCE BAND	BUILDINGS	BIAS (OURS - THEIRS)	MAE	R
> 0.00 (raw outline)	585	-1.15 m	2.34 m	+0.269
> 0.50	581	-3.94 m	4.30 m	+0.336
> 0.70	392	-5.35 m	5.49 m	+0.418
> 0.85 (confident)	168	-6.94 m	7.04 m	+0.452

-6.94 M IS THE HONEST HEADLINE, NOT -1.15 M

Open Buildings' low-confidence outlines are dilated blobs that spill onto surrounding ground and canopy, which drags the disagreement toward zero. The larger, less flattering number comes from the footprints it is most sure about. Correlation *improves* as the threshold rises, so this is not noise thinning out — it is a real, monotone effect.

And it is consistent with our known behaviour: we under-call tall buildings, and Sikkim's hill town is dense and vertical. **Two independent datasets, on two continents, against two different references, now say the same thing.**

Do not quote -1.15 m as the India result. Anyone who knows the dataset will ask which band was thresholded.

Terrain leakage on 30-degree slopes: tested, and clean

AGL is terrain-removed by definition, so a correct model reads the same on a rooftop at 1,200 m as on one at 400 m. Flat Jacksonville and Omaha can never test that. Copernicus GLO-30 over this crop gives **947 m of relief at a median slope of 31.4°**.

CORRELATION	R	EFFECT SIZE
AGL vs slope	-0.044	-0.005 m per degree
AGL vs elevation	+0.538	+0.0026 m per metre

No slope leakage. The ground's tilt is not being read as height, which is the failure we most feared and the one that would have invalidated the whole hilly-terrain claim. The elevation term correlates but is tiny in effect — 2.5 m of drift across the entire 947 m of relief — and it is confounded, because in the Himalaya vegetation and settlement both change with altitude. Report it as a bounded association, not a proven bias.

Three method choices that change the answer

- **Per building, not per pixel**, because their footprint edges are Sentinel-2 blurry and a pixel score would mostly measure their blur.
- **Confidence intervals resample buildings, never pixels.**
- **A global shift is recovered by FFT phase correlation first**, because at 26° off-nadir a roof sits about 5 m from its own footprint while theirs come from near-nadir Sentinel-2. The recovered shift helps (r +0.436 without, +0.537 with) and its magnitude, 7.2 m, is the right

order — but the correlation peak is only **0.041** against 0.40 on a synthetic self-test, so it is not a confident registration. Both numbers are reported together, never the better one alone.

EROSION WAS A HYPOTHESIS, AND IT WAS WRONG

Shrinking their footprint was expected to strip a blur halo and move the bias toward zero. Measured: -1.92 m at 0 m erosion, -2.72 at 2 m, -3.14 at 3 m — the wrong direction, because erosion drops small components first and so selects for the taller buildings we under-call most. The default is 0 and the sweep prints.

9.11 Standalone deployment, verified

Exported 28 August 2026 from the shipping checkpoint with `tools/export_onnx.py --ckpt checkpoints/run02/best.pt --quantize` , with agreement against PyTorch checked rather than assumed:

	ONNX FP32	ONNX INT8
Size	100,672,732 bytes (100.7 MB), self-contained	36,814,825 bytes (36.8 MB) – 63% smaller
Maximum divergence vs PyTorch	0.0010 cm height, 0.0006 cm sigma, over 804,972 inputs	0.161 m height
Verdict	PASS at a 5 cm tolerance	Lossy by construction — a real trade, not a free win

CPU INFERENCE IS 509 MS PER 518×518 TILE

0.53 megapixels per second in a single ONNX Runtime session. The model runs with **no GPU at all**, which is the claim that matters for a deployable module.

The export is fixed at **518×518**. Dynamic shapes fail at runtime — 546×546 raises a `RUNTIME_EXCEPTION` . This is acceptable because `infer.py` is sliding-window at exactly 518, but a `dynamic_axes` argument in the export call would imply a flexibility that is not there.

The single-file viewer was verified by loading it, not by inspecting it

28 August 2026, headless Chrome against `file:///.../viewer_standalone.html` — the path a judge takes when they unzip the submission and double-click:

CHECK	RESULT
External <code>src</code> / <code>href</code> references	None — nothing to block under <code>file://</code>
Leftover ES <code>import</code> statements	0 — the module rewrite holds
Embedded model	<code>run02</code> , with zero mentions of <code>zeroshot</code>
Renderer actually started	Source has 0 <code><canvas></code> tags; the rendered DOM has 1 , tagged <code>data-engine="three.js r169"</code> at 764×429

The canvas is the proof: it does not exist in the file and is created only if the inlined Three.js executes and WebGL initialises. Verified under SwiftShader software rendering, so a machine with a real GPU is strictly more capable than what this test passed on.

Not verified: that every scene's geometry and textures are visually correct. That needs a human looking at it, and it is the one deployment check still owed.

The 3D viewer

Half the marks. The design rules it is built to, every feature, every label, and how it becomes a single double-clickable file.

10.1 What is being scored

ISRO's wording: “projection accuracy, visual fidelity, navigability of the flythrough, interface intuitiveness, stability, and successful standalone deployment.” That is 50% of the total marks.

10.2 The three rules that outrank every feature

1. Usable without explanation	A juror should understand what they are looking at in five seconds, with no caption, no legend to study, and no term they have to already know. If a control needs explaining, the label is wrong. This does not mean hiding the rigour: the plain word goes on the button and the precise number goes in the readout. “How sure we are, ±2.1 m” beats “sigma (m)” and carries strictly more information.
2. No decoration that carries no information	No purple gradients, no glassmorphism, no emoji headings, no hero section, no controls that exist to look busy. The jury are Space Applications Centre image-processing specialists who have used QGIS for twenty years. The data is the only saturated thing on screen; the interface stays near-neutral. Never a rainbow or jet colormap — it is the classic amateur signal in remote sensing.
3. Light theme	It survives a projector in a demo hall, screenshots drop into a submission PDF without inverting, and it puts the viewer in the same visual register as QGIS and ArcGIS Pro rather than a game engine.

Two things a light background forces, both done: the canvas is `#eef1f4` and never pure white, or the surface silhouette disappears and every edge glares; and the ramps run light→dark rather than dark→bright, or the low end dissolves into the page. All three ramps are monotonic in lightness, so they survive greyscale printing. Ambient light is set much stronger than a dark scene needs (hemisphere 1.15) or the terrain reads as a dark blob cut out of the page.

10.3 Label discipline — the actual translations

JARGON AVOIDED	WHAT THE INTERFACE SAYS
sigma / σ (m)	How sure we are — “±2.1 m”
Signed error vs ground truth	Where we're wrong — “blue = we said too low, red = too high”
Vertical exaggeration ×3	Height boost ×3
nDSM / AGL	Height above ground
GSD 0.31 m	Each pixel is 31 cm on the ground
Hillshade azimuth	Sun direction

Other labels visible in the interface: Satellite image · Height above ground · Height above sea · Steepness · Our model · Real LiDAR · Compare with real LiDAR · Checked against LiDAR · No LiDAR here · Error per pixel · Error per building · Tallest we found · Tallest really there · Typical confidence · The ground underneath · Rise across scene · Area covered · Distance on ground · Height difference · Straight-line distance · Mesh lines · Reset view · Fly-through · Upload a satellite image · Your own image.

10.4 Features

Four display modes	Satellite drape (the original photo wrapped over the 3D surface), height ramp (sequential blue), uncertainty (sequential warm), slope. A fifth, error, appears only where LiDAR truth exists — diverging blue/pale/red centred on zero.
--------------------	---

Drag-to-compare	A divider the user drags between our surface and real LiDAR. The clip plane is rebuilt from the camera every frame, so the divider keeps meaning the same thing while the user orbits. This is the most convincing five seconds available — the two pictures either match or they do not — and it is literally the “validation” half of the accuracy criterion.
Where we're wrong	The error map. LiDAR voids are greyed rather than reported as zero error.
Measurement	Two clicks give ground distance, height difference and straight-line distance in metres, with a <i>calibrated</i> error bar on the height difference (Part 9.8).
Sun direction	Hillshade azimuth slider, default 315° — north-west, the cartographic convention. Lighting from the lower right inverts relief perception.
Height boost	Vertical exaggeration slider, starting at 1.5×. See the honesty note below.
Navigation	Click to look, WASD to move, Q / E down and up, Shift to sprint, Esc to release the pointer.
Auto tour	A cinematic orbit path, used for the demo video.
Other	Wireframe toggle, reset view, legend, statistics panel, frames-per-second readout, grid size readout, upload panel.

WHY HEIGHT BOOST EXISTS, AND WHY IT IS NOT CHEATING

A typical scene covers about 300 m of ground but its buildings are only 5–30 m tall. Without exaggeration the 3D view looks almost completely flat and nothing is visible.

Three things make it honest: it stretches the **picture**, never the data — the saved height file records no stretching at all; the measuring tool **removes the stretch** before reporting any number; and the real LiDAR reference is stretched by the same amount, so any comparison stays fair. The slider is labelled on screen and the panel says “the numbers stay true”. Dragging it to 1.0 shows true proportions.

The strongest follow-up: stretching cannot be hiding anything, because we predict buildings too *short*, not too tall. On the median urban test tile the tallest real building is 93.6 m and we say 33.9 m — even stretched by 1.5, it still appears shorter than it really is.

10.5 The scenes that ship

SCENE	SIZE	GSD	LANDSCAPE	LIDAR TRUTH
Mixed — Jacksonville (default)	1024×1024	0.3 m	mixed	yes
Urban — Omaha	1024×1024	0.3 m	urban	yes
Sparse — Omaha	1024×1024	0.3 m	sparse	yes
Forested — Jacksonville	1024×1024	0.3 m	forested	yes
Hilly — Sikkim, India	2048×2047	0.977 m	hilly	no
Detail — Sikkim town	1638×1637	0.305 m	detail	no

The scene picker deliberately mirrors the four landscapes ISRO names. Labels put the terrain first — “Urban — Omaha”, not “Omaha — urban” — because “urban” tells a juror what they are about to look at and maps onto the stated criterion, while the place name does neither.

Sikkim has no LiDAR, so the compare tool and the error map **disable themselves** and the panel says so, falling back to the Open Buildings cross-check labelled as a satellite estimate rather than truth.

Sikkim needed the mountain putting back

The model predicts height above ground, so the raw Sikkim output ranges 0–17.9 m: the Himalaya is exactly the thing AGI removes. Rendered on its own it is a flat plane with buildings on it, which answers nothing about hilly terrain.

Fixed by fetching Copernicus GLO-30 for the tile, cubic-resampling it onto the 0.305 m crop grid, and shipping it as `terrain.bin`. The viewer stacks our heights on top: **248 m of relief across 500 m at a mean slope of 24°**, with our buildings standing on it. That is a real DSM — ground plus everything on it.

Two rules keep it honest. **Colour still comes from the above-ground value, never the mesh Y coordinate**, or the height ramp silently becomes an elevation map. And the statistics panel names the provenance: the mountain is Copernicus at 30 m posts, everything standing on it is ours.

10.6 The scene bundle format

Each scene directory holds raw binaries plus a manifest:

FILE	CONTENTS	SIZE FOR 1024×1024
<code>height.bin</code>	float32 metres above ground, row-major	4,194,304 bytes
<code>sigma.bin</code>	float32 per-pixel uncertainty in metres	4,194,304 bytes
<code>truth.bin</code>	float32 LiDAR reference, where it exists	4,194,304 bytes
<code>terrain.bin</code>	float32 bare-earth elevation, where anchored	varies
<code>texture.jpg</code>	the original photograph, draped	~280 KB
<code>manifest.json</code>	everything below	~2 KB

WHY RAW FLOAT32 AND NOT A PNG

Browsers decode images through an 8-bit canvas. A 16-bit PNG of heights would be silently quantised on the way in, and the metres would stop being metres. Raw `Float32Array` over `fetch` is the only transport that preserves them exactly.

What the manifest carries

Provenance and geometry: the model that produced it (`"run02 + TTA"`), the CRS and affine transform, the bounds, pixel size and its units, the ground sample distance *and where that GSD came from* (`"gsd_source": "--gsd flag"`), the extent in metres, the vertical exaggeration baked in, and how many void pixels were filled.

Statistics the panels display: height minimum and maximum, sigma minimum, maximum and mean, truth minimum and maximum, per-pixel RMSE / MAE / bias / 95th-percentile absolute error, building and ground per-pixel RMSE, the building pixel count, and a full `building_wise` block with the number of buildings, RMSE, MAE, bias, median absolute error, correlation, and the truth median and 90th-percentile heights.

Each numeric block carries a note explaining what it is comparable to. For example the per-pixel error note reads: *“per-pixel, whole tile. Roof edges dominate; per-building error is the comparable figure.”*

10.7 The standalone build

`tools/build_standalone.py` inlines everything into one file: `viewer_standalone.html`, **15,156,848 bytes (15.16 MB)**. Nothing in `main.js` changes.

Three.js	Minified and ending in a single <code>export{a as Vector3,...}</code> . Rewriting that to <code>const THREE={Vector3:a,...}</code> makes it plain inline script.
fetch	Shimmed against an embedded asset table. Because the shim owns the bytes <code>main.js</code> receives, heights are stored uint16-quantised and dequantised on the way out — half the file size for a few millimetres of precision, against a model whose error is measured in metres.
Textures	Routed through <code>THREE.DefaultLoadingManager.setURLModifier</code> , which exists for exactly this.
Scoping	<code>main.js</code> is wrapped in its own block. Inlined, it would share scope with minified Three.js, which declares <code>\$</code> ; the bundle then fails to parse and the page is blank with only a <code>SyntaxError</code> to show for it.

THE DEPLOYMENT BUG THIS CAUGHT

`index.html` loads as an ES module and fetches the scene binaries. **Both are blocked under `file://`**. A judge who unzipped the submission and double-clicked would have seen an empty page — against a criterion that says “successful standalone deployment” in as many words.

`run_viewer.bat` covers it by starting a local HTTP server. The single-file build removes the Python dependency entirely and is what should ship.

Deployment

The whole product runs publicly on an ordinary CPU web host. This part covers what runs where, the measured timings, the guards, and the three constraints that shaped the design.

11.1 What is live

Public URL	https://project5.zaidansari.tech — verified HTTP 200 on 6 September 2026
Platform URL	https://depthwizard-sih2026.azurewebsites.net
Resource group	sih2026-depthwizard
App Service plan	dwz-plan , Linux, SKU B2 — 2 vCPU, 3.5 GB RAM
Region	Central India
First live	6 September 2026
Health endpoint	/healthz — returns 200
Startup command	python tools/serve_app.py

NO GPU ANYWHERE IN THE DEPLOYMENT

Viewer, baked scenes, and the full upload → DSM → 3D pipeline all run on two shared CPU cores. That is a measured claim, not an aspiration.

11.2 Measured timings

SETTING	INFERENCE	WALL CLOCK (INCL. ~8 S MODEL LOAD)	PEAK RSS
Single pass, 9 windows — desktop CPU	5.0 s	13.3 s	2.05 GB
×8 D4 TTA + zoom-2 fusion — desktop CPU	57.2 s	86.2 s	2.21 GB
Fast mode on the live B2 plan	—	60 s POST to served scene	—

Sixty seconds is about 4.5× the desktop's 13.3 s, which is what two shared vCPUs against six local cores predicts. **That is the number to quote, not the desktop one.**

Sixty seconds is usable for “try it yourself” and too slow to stand in front of. Demo from a baked scene; offer upload as the thing a judge does afterwards.

11.3 Why App Service and not containers

Both were tried. On an **Azure for Students** subscription:

BLOCKER	ERROR	CONSEQUENCE
ACR Tasks is barred	TasksOperationsNotAllowed	az acr build cannot build an image in the cloud. Building locally instead needs a Docker daemon.
Container App environments are capped per region	MaxNumberOfEnvironmentsInSubExceeded	Central India's slot is already taken by another project.

Docker cannot run on this machine	<code>VirtualizationFirmwareEnabled: False</code>	SVM Mode is off in the BIOS of the Ryzen 5 5600G. WSL2 and Hyper-V both need it, so there is no software workaround — it is a BIOS trip plus <code>wsl --install --no-distribution</code> as administrator.
-----------------------------------	---	--

App Service's Python runtime sidesteps all three: **Oryx** installs the dependencies on the platform from `requirements.txt`, so nothing local has to build or push an image. `deploy/Dockerfile` is kept anyway — it is correct, and it is the route back to Container Apps on any subscription that permits ACR Tasks.

11.4 Staging — what gets deployed

`python deploy/stage.py` assembles the deployable tree. Total context: **336.3 MB**. Three things dominate the size and each is handled:

Checkpoint stripped	297.1 MB → 99.2 MB. <code>best.pt</code> carries optimiser state, scheduler state, RNG state and the training arguments — everything needed to <i>resume</i> a run and nothing needed to serve one. Kept: <code>model</code> , <code>model_id</code> , <code>height_scale</code> , <code>model_config</code> , <code>epoch</code> , <code>val</code> . Epoch and val are kept because <code>infer.py</code> prints them, and a served result that cannot name its checkpoint is not evidence.
HF cache baked in	<code>model.py</code> builds the backbone with <code>from_pretrained()</code> , which reaches Hugging Face on a cache miss. <code>blobs/</code> is dropped because an HF cache stores every file twice — once under its SHA, once as a snapshot link — and only the snapshot is resolved at load time. <code>HF_HUB_OFFLINE=1</code> turns any surviving download into a loud failure instead of a slow first request.
Uploaded scenes excluded	<code>upload_*</code> scenes are produced at runtime and sit in the same directory as the baked ones. Without the filter the deployment would ship whatever happened to be on the development machine. <code>index.json</code> is pruned to match, since an entry with no directory makes the viewer fetch a missing manifest and presents as the whole picker being broken.

`stage.py` refuses to build a context with zero baked scenes rather than deploying an empty picker. The zip is built at compression level 1 on purpose: the bulk is `.pt`, `.safetensors`, `.bin` and `.jpg`, none of which compress, so level 9 buys a percent or two for several minutes.

11.5 Application settings

SETTING	VALUE	WHY
<code>SCM_DO_BUILD_DURING_DEPLOYMENT</code>	true	Runs Oryx. Without it nothing is installed.
<code>HF_HUB_OFFLINE</code>	1	Never call Hugging Face at request time
<code>DW_WORK</code>	<code>/tmp/depthwizard</code>	Uploads and raw output on local disk, not the slow network share
<code>DW_QUALITY</code>	fast	Default; clients may request <code>accurate</code>
<code>OMP_NUM_THREADS</code>	2	Matches the plan's core count
<code>WEBSITES_CONTAINER_START_TIME_LIMIT</code>	1800	Loading torch is slow on first boot

ORYX RUNS THE APP FROM `/TMP/<HASH>/`, NOT `/HOME/SITE/WWWROOT`

And that hash changes with every deployment. Absolute paths in application settings (`DW_CKPT`, `HF_HOME`) silently point at nothing: the site looks healthy and every upload dies on a missing checkpoint. This actually happened.

The fix is to derive paths from `__file__` instead, which `tools/serve_app.py` now does — and the two absolute app settings were deleted.

`requirements.txt` declares `--extra-index-url https://download.pytorch.org/whl/cpu` in the file itself, because Oryx runs a plain `pip install -r requirements.txt` and never sees command-line flags. Without it, pip resolves the default CUDA wheel — about 2.5 GB of libraries this plan has no GPU to use.

11.6 The public server

`tools/serve_app.py` is a **separate entry point** from `tools/serve_viewer.py` rather than a flag on it. The local tool binds loopback, accepts 400 MB uploads and trusts whoever started it — the right defaults for a tool you run yourself and the wrong ones for a URL. Two files means neither can silently become the other.

Endpoints

GET <code>/healthz</code>	Liveness probe. The platform restarts what it cannot probe.
GET <code>/api/capabilities</code>	Publishes the current limits so a client can warn before spending an upload. Live response on 6 Sep 2026: <code>{"upload": true, "dem": false, "quality": "fast", "max_upload_mb": 50, "max_megapixels": 13.8, "max_side_px": 3713, "sec_per_megapixel": 33.0}</code>
GET <code>/api/job/<id></code>	Polls a running job.
POST <code>/api/upload</code>	Accepts an image, validates it, queues a job, runs inference and export, and registers the resulting scene.
Static	Everything under <code>viewer/</code> , including the baked scene binaries.

11.7 Limits the server enforces

Not decoration: one request costs CPU-minutes, so an unbounded queue is a way to take the site down by being mildly popular.

GUARD	DEFAULT	ENVIRONMENT VARIABLE
Upload size	50 MB	<code>DW_MAX_UPLOAD_MB</code>
Image area	13.8 Mpx ($\approx 3713^2$) in fast mode	derived — see below
Jobs in flight	3, then HTTP 429	<code>DW_MAX_QUEUE</code>
Per-IP spacing	20 s	<code>DW_IP_SPACING</code>
Uploaded scenes kept	12, newest first	<code>DW_SCENE_BUDGET</code>
Job abandoned after	600 s	<code>DW_TIMEOUT</code>

Why an area cap when there is already a byte cap

Because the two are barely related for satellite imagery, and only one of them predicts cost. A Maxar visual strip is **17408×17408 — 303 megapixels — in 12 MB**, sparse and JPEG-compressed inside the TIFF. It passes a 50 MB byte cap comfortably and then needs about **2.8 hours**, so the job used to sit there and time out at 600 s having told the user nothing in the meantime.

The cap is *derived*, not a magic number:

FIXED_S + megapixels * SEC_PER_MPX <= BUDGET_FRAC * DW_TIMEOUT		
FIXED_S	= 25 s	checkpoint load, export, four array writes
SEC_PER_MPX	= 33 s	from the 1024x1024 upload that takes 60 s on this plan
BUDGET_FRAC	= 0.8	
TTA_FACTOR	= 11.4	57.2 s / 5.02 s -- drops the ceiling to ~1.2 Mpx in accurate mode

So the limit moves correctly if the timeout or the plan changes. The refusal is immediate and specific: it names the dimensions, the projected time, and the size to crop to. Reading the header also catches a **truncated or still-downloading GeoTIFF** in the same millisecond, instead of forty seconds into inference with a `TIFFReadEncodedTile` traceback.

A REFUSAL DOES NOT CONSUME THE CALLER'S TURN
Only an accepted job counts against per-IP spacing. Rejecting someone and then making them wait twenty seconds to try a smaller crop would be a worse experience than not having the guard at all.

Baked scenes are never reaped. Per-IP spacing reads `X-Forwarded-For`, which is client-supplied and therefore spoofable — that is fine, because it only paces polite users apart, and the queue cap is what actually bounds the work.

11.8 Custom domain and TLS

TYPE	NAME	VALUE	PROXY
CNAME	project5	depthwizard-sih2026.azurewebsites.net	DNS only
TXT	asuid.project5	the app's customDomainVerificationId	—

```
az webapp config hostname add -g <rg> --webapp-name <app> --hostname project5.zaidansari.tech
az webapp config ssl create -g <rg> --name <app> --hostname project5.zaidansari.tech
az webapp config ssl bind -g <rg> --name <app> --certificate-thumbprint <tp> --ssl-type SNI
```

The grey cloud is required *while the certificate is issued*: Azure's free managed certificate validates by resolving the CNAME to App Service, and Cloudflare's proxy breaks that check. Once issued, the proxy can be turned on with SSL mode Full (strict).

THE SSL CREATE COMMAND REPORTS A JSON PARSE ERROR BUT STILL ISSUES THE CERTIFICATE

It leaves the certificate *unbound*. The fix is to fetch the thumbprint and run `ssl bind` separately. A failure message is not proof that nothing happened.

11.9 Persistence behaviour, measured twice

UPLOADED SCENES SURVIVE AZ WEBAPP RESTART

Measured twice on 6 September 2026, contrary to the assumption written down first. A restart reuses the extracted tree under `/tmp/<hash>/`, so runtime scenes persist.

What *does* clear them is anything that recreates the container — an SSL bind or a redeploy — because the platform extracts a fresh tree and `stage.py` excludes `upload_*`.

So `DW_SCENE_BUDGET` is the only routine bound on disk growth, and a visitor's upload stays publicly listed until twelve more push it out. There is no delete endpoint; if a specific scene must go before then, redeploy.

11.10 Known gaps in the deployment

- **The model reloads on every request**, costing about 8 s of every upload, because `run_job` shells out to `infer.py` exactly as the local tool does. Holding it resident means reimplementing the auto-zoom, fusion and DEM logic inside `infer.py`'s `main()`, and duplicating the inference path is how a served result quietly stops matching the measured one. It stays until `infer.py` exposes a reusable entry point.
- **DEM anchoring is off by default** (`DW_DEM=1` to enable), because `--dem auto` reaches Copernicus during a request that is already slow.
- **The viewer does not yet warn about oversized images** before upload, even though `/api/capabilities` publishes `max_megapixels`. The server refuses correctly; the client could refuse sooner.

Repository map

Every file, what it is for, and how big it is. 19,713 lines of Python across 62 modules, plus the viewer.

12.1 Layout

The repository is `D:\sih2026\depthwizard`. Data, weights, logs, outputs and the virtual environment live *outside* it, under `D:\sih2026\`.

PATH	CONTENTS
<code>D:\sih2026\depthwizard\</code>	The git repository — all source code and documentation
<code>D:\sih2026\data\</code>	Shards, extracted archives, Maxar and Open Buildings caches, uploads, GAMUS shards
<code>D:\sih2026\checkpoints\</code>	run01 through run06 — <code>best.pt</code> , <code>last.pt</code> , <code>train_log.jsonl</code>
<code>D:\sih2026\out\</code>	Inference outputs, evaluation reports, ONNX exports
<code>D:\sih2026\testkit\</code>	10 verified test images: 6 held-out DFC tiles, 3 Nepal crops, 1 Sikkim 500 m crop
<code>D:\sih2026\papers\</code> , <code>logs\</code> , <code>.venv\</code> , <code>.hf\</code>	Literature PDFs, machine vitals logs, the Python environment, the Hugging Face cache

12.2 Core package — `depthwizard/`

FILE	LINES	PURPOSE
<code>model.py</code>	405	DA-V2 backbone + DPT neck + twin (mean, log-variance) head. Also the optional binned head with head-tail cut, the input-size check, parameter groups, and checkpoint reconstruction.
<code>losses.py</code>	304	Gaussian NLL, β -NLL, gradient matching, Chamfer, distribution constraint, head-tail cross-entropy, assembled as <code>CompositeLoss</code> .
<code>metrics.py</code>	289	RMSE/MAE per class <i>and per terrain</i> , per-building statistics, calibration curve, expected calibration error, uncertainty-error rank correlation.
<code>dataset.py</code>	274	Shard loader, two-stage shuffle, D4 and photometric augmentation.
<code>dem.py</code>	424	Copernicus GLO-30 tile discovery and fetch, resampling, absolute-DSM composition, bare-earth approximation, and the GCP fitting functions <code>fit_gcp_offset</code> , <code>fit_gcp_affine</code> , <code>fit_gcp_power</code> .

12.3 Top-level entry points

FILE	LINES	PURPOSE
<code>train.py</code>	622	Fine-tuning. Resumable, bf16/fp16 autodetect, thermal governor, tripwires, append-only JSONL logging.
<code>infer.py</code>	548	Sliding-window inference over a full scene. TTA, zoom fusion, auto-zoom, DEM anchoring, GCP calibration, optional immediate scoring.
<code>requirements.txt</code>	—	Training-time dependencies.
<code>run_viewer.bat</code>	—	Starts a local HTTP server, because the module viewer cannot load from <code>file://</code> .
<code>viewer_standalone.html</code>	15.16 MB	The entire viewer, all scenes, all textures, Three.js, in one double-clickable file.

12.4 Tools — tools/

DATA PREPARATION

<code>prepare_data.py</code> (381)	DFC2019 Track 1: extract → probe → shard. The probe <i>measures</i> what the documentation does not specify.
<code>prepare_gamus.py</code> (567)	GAMUS: list → probe → shard, streaming, keeping no raw data.
<code>link_union.py</code>	Builds <code>data/shards_union</code> as hardlinks to the DFC2019 and GAMUS shards — zero extra bytes.
<code>parse_imd.py</code>	Parses WorldView <code>.IMD</code> metadata into JSON: solar and viewing geometry.
<code>maxar_opendata.py</code> (202)	Maxar Open Data: free 0.3–0.6 m imagery including India, CC-BY-4.0.
<code>open_buildings.py</code> (201)	Google Open Buildings 2.5D Temporal: independent building heights over India.
<code>bhoonidhi.py</code> (257)	Bhoonidhi (NRSC) API client: authenticate, list collections, search, download.
<code>kaggle_push.py</code> (165) / <code>kaggle_watch.py</code>	Push shards to Kaggle as a private dataset; poll a kernel and print one line per state change.

EVALUATION AND ANALYSIS

<code>evaluate.py</code> (416)	Score a checkpoint on a region-disjoint split and write a full report.
<code>zero_shot_baseline.py</code> (236)	How far does stock DA-V2 get with no fine-tuning at all?
<code>compare_runs.py</code> (149)	The ablation table — generated rather than typed.
<code>compare_open_buildings.py</code> (328)	Score a prediction against Google Open Buildings 2.5D over India.
<code>pair_calibration.py</code> (303)	Are the error bars on a height <i>difference</i> honest? Measured, by separation.
<code>view_angle.py</code> (300)	Accuracy against satellite view geometry — the domain-gap number we can actually measure.
<code>make_figures.py</code> (248)	Every figure in the evidence pack, regenerated from one metrics file.
<code>analysis/bin_mode_probe.py</code>	Does the binned head <i>know</i> a building is tall and then average that knowledge away?
<code>analysis/calibration_form.py</code>	Which functional form should GCP calibration use? Affine is measurably wrong.
<code>analysis/compression_compare.py</code>	Compare runs on the metric that shows the tail: the compression slope.
<code>analysis/render_compare.py</code>	Side by side: Maxar RGB, our height, Open Buildings height, on one shared colour scale.
<code>analysis/terrain_leakage.py</code>	Is our AGL tracking the hillside? Correlate the low-frequency prediction against GLO-30.

PROBES — EACH ONE A WRITTEN HYPOTHESIS WITH A FALSIFICATION CONDITION

<code>probe_tall.py</code> (256)	Probe 01: can tall-building supervision move the tail, and does it generalise?
<code>probe_backbone.py</code> (154)	Probe 02: is a larger backbone a better prior for satellite height?
<code>guided_refine.py</code> (123)	Probe 03: sharpen a height map against the image it came from, and score the result.
<code>sharpen_probe.py</code> (126)	Probe 03b: can any post-process turn our ramps back into steps?
<code>gsd_probe.py</code>	Probe 05: does the model survive the resolution ISRO will actually evaluate at?
<code>decompress_probe.py</code>	Probe 06: we compress every height toward the median. Can that be inverted?
<code>shadow_probe.py</code> (264)	Can shadow geometry recover the tall buildings the network cannot?

<code>shadow_ceiling.py</code> (260)	The upper bound on shadow: what would <i>perfect</i> shadow segmentation recover?
<code>lds_probe.py</code>	Would LDS reweighting have room to work here? Measure before writing any training code.
<code>gcp_calibration_probe.py</code> (187)	How many ground control points does a user need before heights are right?
<code>nll_deadlock_probe.py</code>	Why run05 died, measured rather than argued.
<code>probe_browser_inference.py</code> (162)	Can the shipped model run in a browser, with no server doing the work?

EXPORT, VIEWER AND SERVING

<code>export_terrain.py</code> (398)	Turn a predicted height map into a bundle the browser viewer can load.
<code>export_onnx.py</code> (200)	Export to ONNX <i>and verify the export actually agrees with PyTorch</i> .
<code>build_scenes.py</code> (178)	Rebuild the whole viewer scene set, reproducibly.
<code>build_standalone.py</code> (210)	Bake the viewer into a single double-clickable HTML file.
<code>verify_viewer.py</code> (131)	Load the built viewer in a real browser and assert it actually rendered.
<code>serve_viewer.py</code> (245)	Local tool. Binds 127.0.0.1, 400 MB uploads, no throttle.
<code>serve_app.py</code> (506)	The public server. Binds 0.0.0.0, hardened, bounded, rate-limited.
<code>capture_viewer.py</code> (189) / <code>crop_shots.py</code>	Automated screenshots of the viewer for the deck.

OPERATIONS AND SAFETY

<code>preflight.py</code> (200)	Refuse to start a long run that is already broken.
<code>tripwire_selftest.py</code>	Prove the tripwires fire — and prove they do <i>not</i> fire on a healthy run.
<code>vitals.py</code> (194)	Machine vitals watchdog. Logs GPU, RAM and disk continuously; shouts only when it matters. It exists because both hardware failures in this project were sitting in numbers nobody was reading.
<code>thermal_soak.py</code>	Answers “will my 3060 cook itself on an overnight run?”

PRESENTATION

<code>ppt/make_deck.py</code> (515), <code>make_deck_b.py</code> (575), <code>make_deck_c.py</code> (383)	Three generated decks. Shared constants (problem ID, title, theme, team ID) live in <code>make_deck_b.py</code> and are imported by C.
<code>ppt/export_pdf.py</code> , <code>fetch_icons.py</code> , <code>probe_fonts.py</code> , <code>check_layout_b.py</code>	PDF export via LibreOffice, brand icon fetching, font probing, layout assertions.

12.5 Tests — tests/

FILE	LINES	STATUS ON 6 SEP 2026
<code>test_losses.py</code>	245	19 passed, 0 failed. Includes recovery of a known heteroscedastic σ (true 0.50 / 3.00 → recovered 0.497 / 2.99, ratio 6.02 against a true 6.0).
<code>test_gcp_affine.py</code>	178	All 9 passed. Includes exact recovery of a known ratio error — exponent 1.3889 against a true 1.3889, residual 7.74e-07 m — and correct refusal of too-few and short-only control points.

There is no `pytest` in the environment. Each file carries its own `__main__` runner and needs `PYTHONPATH` set to the repository root.

12.6 Documentation — docs/

FILE	LINES	CONTENTS
problem-statement.md	131	The official PS scraped verbatim, plus five deltas against internal assumptions.
evidence-pack.md	318	Every accuracy number, measured on the shipping configuration.
evaluation-protocol.md	202	The four protocols, and the India cross-check in full.
literature.md	213	What the published work says, what it changed, and the binned-head outcome.
deployment.md	184	The full hosting runbook.
viewer-design.md	178	The design rules and the label discipline.
gamus-integration.md	367	Plan, disk budget, pre-registered success criteria, phased runbook.
probe-01 ... probe-06	59–97 each	Tall buildings, guided filter, resolution, GSD, compression.
PLAN.md (root)	~1,060	The plan of record: every design decision and why it was made.
CLAUDE.md (root)	~45	Working rules: search the literature first; never quote a number without its protocol; never change GPU settings mid-run.

Negative results

Everything that was tried and did not work. This is not an appendix — the elimination is itself the finding, and it is what makes the remaining limitation defensible.

HOW THESE WERE RUN

Each hypothesis was written down with a falsification condition *before* the run, then measured. Two of the first five falsified something the team believed.

13.1 The summary table

HYPOTHESIS	RESULT
Guided filtering sharpens our surfaces	Wrong. Worse at every setting (probe 03).
Morphological toggle contrast sharpens them	Wrong. Worse, monotonically (probe 03).
The tall-building tail is a capacity problem	Wrong. It is data scarcity (probe 01).
Terrain slope leaks into height on hills	Ruled out. $r = -0.044$ at 31° median slope.
Coarse-GSD loss is irrecoverable	Wrong. Only 3.6% of it remains after <code>--auto-zoom</code> .

13.2 The tall-building tail, exhaustively

The under-call above 20 m is the one limitation that could not be removed. Every lever internal to the model is now measured and spent.

LEVER	RESULT
Larger backbone (V1-Large, 335 M)	No information. The run diverged to NaN at epoch 6 from a variance head railed at its clamp. Nine GPU-hours, void. Root cause found afterwards: a 666 m initial residual.
Ordinal / binned head	No effect. Compression slope: regression 0.471, bins 0.429, bins+HTC 0.491 — every head compresses by about half.
Head-tail cut (HTC-DC Net, TGRS 2023)	Works, and is not the bottleneck. 91% of predicted probability mass sits above 20 m on tall pixels; roof-versus-ground separation was never the failure.
Decoding by argmax instead of expectation	No tall mode to recover. Argmax sits +1.86 m from the expectation; on 40 m+ buildings the single most likely bin is 26.4 m against 75.6 m of truth.
Ensembling two runs with different profiles	Nothing to average. <code>corr(err_run02, err_run04) = 0.925</code> . No weighting beats the single model.
LDS reweighting (Yang et al., ICML 2021)	Premise does not hold. Pixels above 20 m are 7.47% of building pixels and already carry 78.78% of the squared error — the loss is not ignoring them.
Per-scene GCP calibration	A trade, not a fix. Five control points give −21.3% RMSE on tall tiles but +18% MAE, and degrade 7 of 13 tiles.

WHAT THESE SAY TOGETHER

The model is **not** under-parameterised, **not** mis-decoded, **not** under-incentivised, and **not** held back by its head. It assigns 91% of its probability mass above 20 m on a tall building and still cannot separate 26 m from 76 m.

A nadir view of a rooftop does not contain that information. That is the finding, and it is a stronger claim than “we ran out of time.”

13.3 Shadow — the new cue, taken to its ceiling

The obvious escape is a physical cue, and the metadata for one exists: 67 WorldView-3 `.IMD` files carry solar geometry, sun elevation spanning 23.4°–74.5°. `height = shadow_length × tan(sun_elevation)` is how photointerpreters measured buildings before

computers, and its error profile is the *inverse* of ours — shadow is most precise on exactly the tall buildings where the network fails.

The cue is really there. Marching away from the sun on the tallest building in `OMA_288_012` gives luminance 166 on the roof, 93–113 held for about 120 px, then 254 on lit ground. Marching *toward* the sun leaves the tile immediately.

QUESTION	MEASUREMENT
Would <i>perfect</i> shadow segmentation be enough?	No. Oracle shadows ray-cast from the truth DSM give r 0.503 , slope 0.583, RMSE 2.61 m over 675 buildings. Our network already achieves r 0.787 per building.
How close is real detection to that oracle?	Not close. Luminance + Otsu thresholding scores IoU 0.187 (precision 0.204, recall 0.689).
Is occlusion the limiter on tall buildings?	No. Across the ten tiles richest in tall stock, all 10 components above 20 m had <i>unobstructed</i> shadow paths.
Then what is?	Population. Those ten tiles hold 10 buildings above 20 m out of 766 ; the whole validation set holds about 60.

So the ceiling on the cue sits *below* the model we already have, and the population available to validate any tall-building method is about sixty buildings. That is why the tail stays in Limitations rather than being quietly fixed.

13.4 The binned head, in detail

	RMSE	BLDG RMSE	BLDG BIAS	GRND RMSE	ECE	Σ RANK P
run02 direct regression	6.454	16.37	−4.70	1.92	0.083	+0.836
run03 binned soft-argmax	6.950	17.81	−5.50	1.75	0.106	+0.784

7.7% worse overall and 8.8% worse on buildings — the thing it was built to fix. Calibration got worse too.

The one metric that improved says what happened. Ground RMSE fell to 1.75 m, the best of any run, while buildings degraded. Smoothing helps homogeneous terrain and hurts tall discontinuous structure. That is the over-smoothing signature of soft-argmax, and the distribution supervision added specifically to prevent it did not.

A PROTOCOL NOTE THAT NEARLY CAUSED THE WRONG MODEL TO SHIP

Crop-wise validation disagreed in *direction*: run03 best 7.894 against run02 best 7.980, so crop-wise says run03 *won*. Whole-tile says it lost by 7.7%. Same checkpoints, opposite conclusions. **Anyone reading only `train_log.jsonl` would have shipped the worse model.**

13.5 What the literature changed

A targeted literature search on 27 August 2026 contradicted two decisions already taken:

- **Our failure mode is *the* known failure mode of this field.** HTC-DC Net states it directly: “the distribution of height values is long-tailed... trained networks are usually biased and tend to **underestimate building heights**... predictions can include many fatal cases for higher objects with incredibly large errors.” That is our error profile verbatim.
- **The fix is not loss reweighting.** Every paper that moves the needle reformulates the *output space*. Hours had gone into designing a β -NLL reweighting fix with a two-hour GPU run already committed to it. One search surfaced the paper that named the failure, showed reweighting is the wrong class of fix, gave the method that works, and revealed that published DFC19 numbers are not comparable to ours at all.
- **Soft-argmax needs distribution supervision or it will not help.** A roof edge is exactly a foreground/background bimodal pixel — the truth is either roof height or ground, never the average. Supervising only the mean lets the network put mass on both modes and bleed the edge.

This is why the repository carries a standing rule: **search the literature before spending compute.**

Limitations and unfinished work

Stated plainly. Every one of these should be said out loud before a judge finds it.

14.1 Accuracy limitations

1. **Buildings above 20 m are under-called by about 17 m.** Data scarcity, proven by probe 01. Training data tops out at 82.8 m while validation reaches 155.3 m. The direction is conservative — we are never too tall.
2. **Forested ground carries 2.5 m of error** because a camera cannot see through canopy, while the LiDAR reference can. That is optics, not a model defect.
3. **Tile borders are worse** — outer-pixel RMSE is roughly double the interior. A known property of patch-based vision-transformer inference.
4. **The absolute DSM anchor is itself a surface model.** Copernicus GLO-30, like SRTM, includes buildings and canopy, so dense urban composites double-count part of the building height. `--dem-bare` approximates a correction and says that it is one.
5. **No Indian LiDAR.** Everything over India is cross-checked against another model and is labelled as agreement, never as accuracy.
6. **The test split is unscored.** By design, until the very end, once. It is also materially easier than validation.
7. **Urban RMSE is 13.9 m**, four to seven times the other landscapes — and cities are where disasters concentrate. The honest framing: correlation stays at +0.808, so the shape is right and the scale of tall structures is not; and 94% of the building stock is under 10 m where the error is 1.4 m.

14.2 Coverage limitations

- **Two American cities.** All LiDAR-verified accuracy comes from Jacksonville and Omaha. Both are flat.
- **No hilly LiDAR anywhere.** The hilly case is Sikkim, which has no reference data, so it is a cross-check.
- **GAMUS is not trained on.** The sponsor's recommended dataset is ingested to 200 tiles and blocked on disk. No model in this project has seen a GAMUS tile.
- **Cross-sensor generalisation is untested.** Final evaluation uses ISRO imagery at an unknown resolution. `--auto-zoom` addresses the resolution part, measured on DFC2019 downsamples — not on real ISRO data, which we do not have.

14.3 Engineering limitations and known bugs

ITEM	STATE
<code>infer.py --help</code> crashes	A literal <code>%</code> in one help string breaks argparse's formatting. Cosmetic, but <code>--help</code> is not a way to discover the flags.
Model reloads per request on the server	Costs ~8 s of every upload. Fixing it means <code>infer.py</code> must expose a reusable entry point first.
60 s per upload on the live plan	Usable for “try it yourself”, too slow to demo live.
Viewer does not pre-warn about oversized images	<code>/api/capabilities</code> publishes the limit; the client does not read it yet.
No delete endpoint for uploaded scenes	A visitor's upload stays listed until twelve more push it out.
Docker unavailable on the build machine	BIOS SVM Mode is off. Blocks WSL2 and Hyper-V. Requires a reboot into BIOS.
run06 never finished	No checkpoint exists. run02 from 27 August remains the shipping model.
Demo video not made	The tour path exists; the video does not. The pitch-video QR box on deck slide 6 is still a placeholder.
Visual verification of every scene	The standalone viewer is proven to <i>render</i> ; that every scene's geometry and textures are visually correct still needs a human to look.

14.4 What would be done with more time, in priority order

1. **Finish the GAMUS ingest and train on the union.** It is 36× our tall-building supervision and the only credible route to a generalisation claim. Blocked on about 6 GB of disk.
2. **Hold out a whole GAMUS city** (New York) and report cross-city transfer. That is the number that would answer “why should ISRO believe it works anywhere else?”
3. **Finish the reference-point tool in the viewer.** The calculation already works and is tested; only the click-to-place interface is missing.
4. **Use shadows for tall buildings only.** Shadows are best exactly where the model is worst. It cannot replace the model, but it might correct the specific cases that fail.
5. **Score the test split, once,** for the final reported number — and state that it is easier than validation when doing so.

References and command reference

Every paper, dataset and external source the project depends on, followed by every command needed to reproduce it.

15.1 Papers

HTC-DC Net	Chen et al., <i>IEEE Transactions on Geoscience and Remote Sensing</i> , 2023. arxiv.org/abs/2309.16486 · code at github.com/zhu-xlab/HTC-DC-Net. The central reference. Names the underestimation failure as the field's known problem, gives the head-tail cut and distribution constraint, and supplies the published DFC19 comparison table.
Depth Anything V2	The backbone. Small variant, Apache-2.0, on Hugging Face as <code>depth-anything/Depth-Anything-V2-Small-hf</code> .
DINOv2	The self-supervised vision-transformer backbone inside Depth Anything.
DPT	Dense Prediction Transformer — the decoder architecture. The upsample-after-conv1 ordering in our head mirrors the paper exactly.
Depth Any Canopy	The only published adaptation of Depth Anything to aerial height that reports its hyperparameters. Source of the AdamW / 5e-6 / batch-8 recipe.
Kendall & Gal	The twin-head heteroscedastic uncertainty arrangement.
Seitzer et al.	β -NLL — detach-weighting of the residual term. run02 uses $\beta = 0.5$.
AdaBins	arxiv.org/abs/2011.14141. Adaptive bin widths via softmax; the $\beta = 0.1$ Chamfer convention.
BinsFormer / LocalBins / IEBins	arxiv.org/pdf/2204.00987 · 2203.15132 · 2309.14137. The bin-count ablations that led to N = 128 rather than 256.
Sampling-Gaussian for stereo matching	arxiv.org/pdf/2410.06527, and Adaptive Multi-Modal Cross-Entropy Loss for Stereo Matching, arxiv.org/pdf/2407.07816. The soft-argmax over-smoothing analysis.
LDS / FDS	Yang et al., ICML 2021. Label-distribution smoothing for imbalanced regression — ruled out here by measurement.
TSE-Net	arxiv.org/pdf/2511.13552. Recent context in the same lineage.
GlobalBuildingAtlas	Source of the 5.9 m per-building RMSE benchmark over Asia.

15.2 Datasets and data services

DFC2019 Track 1 (US3D)	ieee-dataport.org/open-access/data-fusion-contest-2019-dfc2019. Baselines and metrics: github.com/pubgeo/dfc2019 (MIT).
GAMUS	huggingface.co/datasets/earthflow/GAMUS, CC-BY-4.0. Dataloader: github.com/EarthNets/RSI-MMSegmentation.
Copernicus GLO-30	<code>https://copernicus-dem-30m.s3.amazonaws.com</code> — open, no account.
Google Open Buildings 2.5D Temporal	Sentinel-2 derived building heights, field <code>building_height</code> in $[0, 100]$ m.
Maxar Open Data	CC-BY-4.0. Event folder <code>India-Floods-Oct-2023</code> , acquisitions dated 2022-03-14.
Bhoonidhi (NRSC)	ISRO's data portal. Client implemented; open products not at a useful resolution for building heights.

ISRO reference repository	github.com/IMG-PROCESS-SAC/SIH-DepthWizard-2026
Problem statement portal	sih.gov.in/sih2026PS

15.3 Command reference

ENVIRONMENT

```
python -m venv .venv
.venv/Scripts/pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
.venv/Scripts/pip install -r requirements.txt
```

DATA

```
python tools/prepare_data.py extract path/to/Train-Track1-RGB.zip path/to/Train-Track1-Truth.zip
python tools/prepare_data.py probe # MEASURES what the DFC2019 docs do not specify
python tools/prepare_data.py shard

python tools/prepare_gamus.py probe --sample 6
python tools/prepare_gamus.py shard --workers 16 # resumable
python tools/prepare_gamus.py holdout # the held-out DC block
python tools/link_union.py --check && python tools/link_union.py
```

BASELINE, TRAINING, EVALUATION

```
# what does doing nothing cost? establish this before training anything
python tools/zero_shot_baseline.py --fit-tiles 40 --eval-tiles 80

python train.py --epochs 12 --batch 8 --workers 4 --max-temp 82

python tools/evaluate.py --ckpt D:/sih2026/checkpoints/run02/best.pt --tiles 80
python tools/evaluate.py --ckpt <ckpt> --split val
python tools/evaluate.py --ckpt <ckpt> --split gdc --corpus gamus
```

THE EXACT COMMAND THAT PRODUCED EVERY SHIPPING NUMBER

```
python tools/evaluate.py --ckpt checkpoints/run02/best.pt --tta \
--fuse-zoom 2 --fuse-sigma 8 --out out/eval_run02_ship
python tools/make_figures.py --metrics out/eval_run02_ship/metrics.json
```

INFERENCE AND EXPORT

```
python infer.py --image scene.tif --ckpt checkpoints/run02/best.pt --out out/scene \
--tta --fuse-zoom 2 --auto-zoom --dem auto

python tools/export_terrain.py --height out/scene.height.tif --sigma out/scene.sigma.tif \
--texture scene.tif --gsd 0.3 --out viewer/scenes/scene

python tools/export_onnx.py --ckpt checkpoints/run02/best.pt --quantize
python tools/build_standalone.py
python tools/verify_viewer.py
```

VIEWER AND SERVING

```
python -m http.server -d viewer 8080 # http://localhost:8080
run_viewer.bat # the same, for a double-click
python tools/serve_viewer.py # local, 127.0.0.1, no throttle
python tools/serve_app.py # public, 0.0.0.0:8080
python tools/serve_app.py --dem # with Copernicus anchoring
```

DEPLOYMENT

```
python deploy/stage.py
az webapp deploy -g sih2026-depthwizard -n depthwizard-sih2026 \
--src-path dwz-app.zip --type zip --async true
```

```
PYTHONPATH=D:/sih2026/depthwizard python tests/test_losses.py
PYTHONPATH=D:/sih2026/depthwizard python tests/test_gcp_affine.py
```

15.4 Viewer controls

INPUT	ACTION
Click	Capture the pointer and look around
W A S D	Move horizontally
Q / E	Move down / up
Shift	Sprint
Esc	Release the pointer
Measure, then two clicks	Ground distance, height difference with ±, straight-line distance

Glossary

Every term and abbreviation used anywhere in this document or in the project, in plain English.

HEIGHT AND TERRAIN

DSM	Digital Surface Model. A map of how high everything is, including buildings and trees. This is what ISRO asked for.
DEM	Digital Elevation Model. A general term for height data.
DTM	Digital Terrain Model. The bare ground, with buildings and trees removed.
rDSM	Relative DSM. The shape of the surface with no absolute datum. What we produce for a photo with no location data.
AGL / nDSM	Above Ground Level, also called a normalised DSM. How tall something is compared to the ground right underneath it. A 10 m building on a 100 m hill is 10 m AGL. This is what our model predicts.
Vertical datum	The zero surface elevations are measured from — roughly, sea level.
Vertical exaggeration	Stretching heights in the 3D view so the shape is visible. Ours starts at 1.5× and is removed before any measurement is reported.
Relief	The total height range across a scene. Sikkim's 2 km crop has 947 m of relief.

IMAGING

GSD	Ground Sample Distance. How much ground one pixel covers. 0.3 m GSD means each pixel is 30 cm across. Smaller means sharper.
Nadir	Looking straight down. What satellites usually do, and why depth models struggle — there is no horizon or perspective to judge scale from.
Off-nadir	Looking at an angle. At 26° off-nadir a roof sits about 5 m from its own footprint.
Monocular	Using one image only.
Stereo	Using two photographs from different angles to work out height. The normal method, and the one being replaced.
LiDAR	A laser scanner, usually flown on an aircraft. Measures distance by timing a light pulse. Our source of correct answers.
InSAR	Interferometric Synthetic Aperture Radar. Radar-based elevation measurement, named in the problem statement.
Photogrammetry	Measuring real-world sizes and distances from photographs.
Orthoimage	An aerial image corrected so that every pixel is viewed as if from directly above.
Georeferenced	A photo that knows where on Earth it was taken.
GeoTIFF	An image file with location information built into it.
CRS	Coordinate Reference System. The definition of what the coordinates in a file mean.
COG	Cloud-Optimised GeoTIFF. Structured so a client can fetch just the part it needs over HTTP.
Megapixel	One million pixels. This determines processing time — not the file size.

Neural network	A program that learns patterns from examples instead of following written rules.
Backbone	The part of the model that understands the image. 89% of ours.
Neck	The part that turns the backbone's coarse output back into a dense per-pixel feature map. 10.9% of ours.
Head	The small final part that turns features into the answer. 0.1% of ours, and the part we modified.
Vision transformer (ViT)	A network that cuts an image into small squares and lets every square take account of every other square. Ours uses 14×14 pixel squares.
Token / patch	One of those squares, and the vector representing it.
DINOv2	The specific backbone inside Depth Anything. Learned from images with no human labelling, which is why it works on many kinds of picture.
DPT	Dense Prediction Transformer. Turns a coarse grid of tokens back into a full-detail answer for every pixel.
Fine-tuning	Taking a model someone else trained and continuing to train it on your own data. This is what makes our model output metres.
Zero-shot	Using a model with no extra training at all.
Checkpoint	A saved file containing a trained model.
Epoch	One full pass over the training data.
Step	One batch of examples, one weight update. run02 took 11,532 of them.
Learning rate	How large a change each step makes.
Weight decay	A pull toward smaller weights, which discourages memorisation.
AdamW	The optimisation algorithm used here.
Overfitting	Memorising the training examples instead of learning the general pattern.
Region-disjoint split	Splitting data by geographic area so no place appears in both training and testing.
Augmentation	Showing the model transformed copies of each example. D4 is the eight symmetries of a square.
TTA	Test-Time Augmentation. Running the same image through several times, flipped and rotated, then averaging. Slower but more accurate.
Sliding window	Processing a large photo as overlapping small squares, because the model only accepts a fixed size.
Ill-posed	A problem with more than one possible correct answer.
Heteroscedastic	Uncertainty that varies from place to place, rather than one number for the whole image.
NLL	Negative Log-Likelihood. The training objective: how surprised the model is by the true answer, given the distribution it predicted.
Soft-argmax	Turning a distribution over bins into a single continuous number by taking its expectation.
Ensemble	Averaging several models. Only helps when they make different mistakes; ours make the same ones (r = 0.925).
ONNX	A standard file format for neural networks. Lets our model run without installing PyTorch.
Quantisation	Storing the model's numbers with less precision to shrink the file. Ours saves 63% of the size and costs 0.161 m of accuracy.

bf16 / fp16 / fp32	Number formats of decreasing speed and increasing precision. bf16 needs Ampere or newer.
---------------------------	--

METRICS

RMSE	Root Mean Square Error. Square every mistake, average, take the square root. Because of the squaring, a few big mistakes push this up a lot.
MAE	Mean Absolute Error. The plain average mistake. Always smaller than RMSE. A big gap between the two means a few large mistakes dominate.
Bias	Whether you are consistently too high or too low. Ours is 17 m too low for buildings over 20 m.
Correlation (r)	How well predictions follow the real pattern, from -1 to +1. High correlation with bad RMSE means the shape is right but the scale is wrong.
Calibration	Whether the model's stated confidence matches how often it is actually right.
ECE	Expected Calibration Error. How far the stated confidence is from the observed frequency. Ours is 0.077.
Sigma (σ)	The model's own statement of how unsure it is about a pixel, in metres.
Rank correlation	Whether the <i>ordering</i> matches, ignoring exact values. Ours between σ and error is +0.829.
IoU	Intersection over Union. How well two shapes overlap. Shadow detection scores 0.187.
Oracle	A pretend perfect version of something, used to find the best a method could possibly do. An oracle result is never deployable.
Compression slope	How much the model squashes the height range. 1.0 would be perfect; ours is 0.471.

PROJECT AND INFRASTRUCTURE

GCP	Ground Control Point. A place where the true height is already known, used to correct a scene.
SRTM	A free worldwide elevation map from a Space Shuttle mission in 2000. Named in the problem statement.
Copernicus GL0-30	The free European worldwide elevation map we use instead. Same 30 m detail as SRTM, no account needed, and it covers the far north where SRTM has gaps.
CartoDEM	ISRO's Indian elevation map. Free only to Indian government bodies, so not available to us.
FABDEM	A true bare-earth product. Non-commercial licence, so not used.
DFC2019	Our training dataset. Aerial photos of Jacksonville and Omaha at 0.3 m per pixel, with laser-measured heights.
GAMUS	The dataset ISRO's own page recommends. Philadelphia, New York and Washington DC — all dense cities.
Mesh	The 3D surface built from the height numbers.
Texture drape	Painting the original photo onto that 3D surface so it looks real.
Hillshade	Artificial shading from a chosen sun direction, which makes relief legible.
Oryx	Azure's build system. Installs Python dependencies on the platform — and runs the app from <code>/tmp</code> , not <code>wwwroot</code> .
SNI	Server Name Indication. The TLS binding mode used for the custom domain.
ACR Tasks	Azure Container Registry's cloud image builder. Barred on Azure for Students.
SVM Mode	AMD's hardware virtualisation switch in the BIOS. Off on this machine, which is why Docker cannot run.

Provenance. Every figure in this document was re-verified against the repository, the checkpoints, the live deployment or a fresh test run on **6 September 2026**. Accuracy figures come from the exact configuration running on the live website — run02 with ×8 D4 test-time augmentation and zoom-2 fusion — scored on 80 held-out whole tiles, 11,983,760 valid pixels and 3,090 buildings, against airborne LiDAR. Where no LiDAR reference exists, results are described as comparisons rather than as accuracy.

DepthWizard · SIH26175 · Team Dev Up, Team ID 47 · Live at project5.zaidansari.tech